

Life-Link Blood Donation App



Session: (2022-2026)

Submitted By:

Muhammad Usman	084597
Hadia Akram	084582
Abu Bakkar Khalid	084586

Supervised by:
Prof.Irfan Shabbir

**DEPARTMENT OF INFORMATION TECHNOLOGY GOVT.
M.A.O GRADUATE COLLEGE LAHORE**

DECLARATION

We hereby declare that this software, neither whole nor as a part has been copied out from any source. It is further declared that we have developed this software and accompanied report entirely on the basis of our personal efforts. If any part of this project is proved to be copied out from any source or found to be reproduction of some other, we will stand by the consequences. No portion of the work presented has been submitted of any application for any other degree or qualification of this or any other university or institute of learning.

Signature: -----

Usman Shafiq [084597]

Signature: -----

Hadia Akram [084582]

Signature: -----

Abu Bakkar Khalid [084586]

CERTIFICATE OF APPROVAL

It is to certify that the final year design project (FYDP) of BS-IT “**Life - Link Blood Donation App**” was developed by

Usman Shafiq 084597, Hadia Akram 084582, Abu Bakkar Khalid 084586 under the supervision of “**Prof Irfan Shabbir**” in my opinion; it is fully adequate, in scope and quality for the degree of Bachelors of Science in **Information Technology**

Signature: -----

FYDP Supervisor:

Signatures (Faculty Advisory Committee (FAC))

Signatures			
Name			
	FAC1	FAC2	FAC3

Signature: -----

Head of FYDP Coordination Office:

Signature: -----

Dated:

Chairperson, Department of Information Technology

Executive Summary

The Life-Link Blood Donation System is a digital platform developed to improve the efficiency and reliability of blood donation and management in Pakistan. The main objective of the project is to provide a unified digital solution that connects blood donors and patients, reducing delays in finding compatible blood during medical emergencies and improving overall coordination within the healthcare system. The system is implemented using a cross-platform architecture, including a React Native mobile application for donors and patients, and a Next.js web dashboard for administration. The backend is built with NestJS and Supabase to ensure secure authentication, real-time data synchronization, and reliable storage. A rule-based donor eligibility and matching module using filtering through database recommends suitable donors based on blood type, location, and donation history, while real-time notifications and bilingual support (Urdu and English) enhance accessibility and responsiveness. The project concludes that integrating with real-time digital systems can significantly improve blood donation processes. Life-Link demonstrates improved donor matching, better blood tracking and faster emergency response, highlighting its potential to serve as a scalable and impactful solution for strengthening healthcare services in Pakistan.

Keywords: Blood Donation System, Healthcare Technology, Mobile Application.

Acknowledgement

We express our gratitude to our Almighty Allah, and after that, we are thankful to our families, who allowed us to get higher education, and we became capable of creating Something productive. We're also thankful to our project advisor and department faculty for their support.

Signature: -----

-

Usman Shafiq [084597]

Signature: -----

Abu-Bakkar Khalid [084586]

Signature: -----

Hadia Akram [084582]

Abbreviations

Table of all the abbreviations and acronyms used in your FYP along with their respective brief description. The abbreviation list must be ordered alphabetically.

Abbreviation	Description
API	Application Programming Interface enabling communication between system components
BL	Backlog List: A list of requirements and features under consideration
DB	Database used to store system data securely
FCIT	Faculty of Computing and Information Technology
FR	Functional Requirement defining system behavior and features
FYP	Final Year Project
NFR	Non-Functional Requirement describing system quality attributes
RN	React Native framework used for mobile application development
SDS	Software Design Specification document
SRS	Software Requirements Specification document
UI	User Interface of the application
UX	User Experience focusing on usability and accessibility

Table of Contents

Contents

<i>Chapter 1 Introduction</i>	<i>13</i>
1. Error! Bookmark not defined.	
1.1	14
1.2 Error! Bookmark not defined.	
1.3 Error! Bookmark not defined.	
1.4 Error! Bookmark not defined.	
1.5	16
1.6 Error! Bookmark not defined.	
1.6.1 Error! Bookmark not defined.	
1.6.2 Error! Bookmark not defined.	
1.6.3 Error! Bookmark not defined.	
1.7 Error! Bookmark not defined.	
1.8 Error! Bookmark not defined.	
1.9 Error! Bookmark not defined.	
1.9.1 Error! Bookmark not defined.	
1.9.2 Error! Bookmark not defined.	
1.10	19
1.11	19
1.11.1	19
1.11.2	20
1.12	20
1.13	21
<i>Chapter</i>	22
2.	23
2.1	23
2.2	23
2.2.1 Error! Bookmark not defined.	
2.2.2	24
2.3	24
2.3.1 Error! Bookmark not defined.	
2.3.2	25
2.3.3	26
2.3.4	26
2.3.5	27
2.3.6	28
2.3.7	28

2.3.8	29
2.3.9	29
2.4	30
2.4.1	Error! Bookmark not defined.
2.4.2	30
2.4.3	30
2.4.4	31
2.4.5	31
2.4.6	31
2.5	31
2.5.1	Error! Bookmark not defined.
2.5.2	32
2.5.3	32
2.5.4	32
2.6	32
2.6.1	Error! Bookmark not defined.
2.6.2	33
2.6.3	34
2.6.4	36
2.6.5	36
2.6.6	37
2.6.7	38
2.6.8	39
2.6.9	40
2.6.10	41
2.6.11	42
2.7	45

Chapter 3

46

3.	47
3.1	Error! Bookmark not defined.
3.1.1	47
3.1.2	48
3.1.3	48
3.2	49
3.2.1	Error! Bookmark not defined.
3.2.2	51
3.2.3	55
3.3	57
3.3.1	Error! Bookmark not defined.
3.3.2	58
3.3.3	58
3.4	59
3.4.1	Error! Bookmark not defined.
3.4.2	61
3.4.3	62

3.5	65	
3	Error! Bookmark not defined.	
3.5.1	Error! Bookmark not defined.	
3.6	69	
3.7	69	
Chapter 4		75
Implementation		75
4. Error! Bookmark not defined.		
4.1 Algorithm:		76
4.2	74	
4.3 Code Repository		79
4.4 Summary		79
Chapter 5		80
Testing and Evaluation		80
5. Error! Bookmark not defined.		
5.1 Error! Bookmark not defined.		
5.2 Error! Bookmark not defined.		
5.3	83	
5.4	86	
Chapter 6		89
System Conversion		89
6	88	
6.1 Conversion Method		90
6.1.1 Pilot Conversion:		90
6.2.1	89	
6.2.2	90	
6.2.3 Training		92
6.3	91	
6.3.1	91	
6.4	92	
6.5	92	
Chapter 7		95
Conclusion		95
7.	94	
7.2	95	
7.3	96	

List of Figures

Figure 1 Gantt Chart	21
Figure 2 Use Case Diagram	44
Figure 3 Class Diagram	51
Figure 4 Sequence Diagram Auth Flow	52
Figure 5 Sequence Diagram Create Blood Request	53
Figure 6 Sequence Diagram Check eligibility	54
Figure 7 Sequence Diagram Request fulfill flow	55
Figure 8 Sequence Diagram Notification flow	56
Figure 9 Sequence Diagram Appointment flow	57
Figure 10 Sequence Diagram Update profile flow	Error! Bookmark not defined.
Figure 11 Sequence Diagram Manage User flow	59
Figure 12 State Transition Diagram	60
Figure 13 Architecture Style	61
Figure 14 UML component diagram	62
Figure 15 Process flow diagram	63
Figure 16 Database Design scheme	65
Figure 17 Auth Screens	71
Figure 18 Request Screens	72
Figure 19 User's Screens	73

List of Tables

Table 1.1 Related System Analysis with Proposed Project Solution	17
Table 1.2 Tools and Technologies for Proposed Project	19
Table 1.3 Week-wise Project Task Schedule and Milestones	20
Table2.1: User Classes and Characteristics	23
Table2.2: Description of FR-1	24
Table2.3: Description of FR-2	25
Table2.4: Description of FR-3	25
Table2.5: Description of FR-4	26
Table2.6: Description of FR-5	27
Table2.7: Description of FR-6	Error! Bookmark not defined.
Table2.8: Description of FR-7	27
Table2.9: Description of FR-8	28
Table2.10: Description of FR-9	29
Table 2.11: Description of FR-10	29
Table2.12: Description of Use-case 1	32
Table2.13: Description of Use-case 2	33
Table2.14: Description of Use-case 3	34
Table2.15: Description of Use-case 4	35
Table2.16: Description of Use-case 5	36
Table2.17: Description of Use-case 6	Error! Bookmark not defined.
Table2.18: Description of Use-case 7	37
Table2.19: Description of Use-case 8	38
Table2.20: Description of Use-case 9	39
Table2.21: Description of Use-case 10	40
Table2.22: Description of Use-case 11	41
Table3.1: System Architecture Layers and Their Responsibilities	61
Table3.2: System Terminology and Data Dictionary	66
Table3.3: User Class Attributes and Methods	68
Table3.4: Donor Class Attributes and Methods	68
Table3.5: Patient Class Attributes and Methods	68
Table3.6: Blood Bank Class Attributes and Methods	69
Table3.7: Admin Class Attributes and Methods	69
Table3.8: Blood Request Class Attributes and Methods	69
Table3.9: Donation Class Attributes and Methods	70
Table3.10: Appointment Class Attributes and Methods	70
Table3.11: Notification Class Attributes and Methods	70

Chapter 1

Introduction

1. Introduction

This chapter introduces the Life-Link Smart Blood Donation System and explains the need for a digital solution for blood donation in Pakistan. It covers the problem, proposed solution, objectives, scope, and system design, and briefly reviews existing systems to show why Life Link is important for solving blood shortage issues.

1.1 Background

Blood donation plays a vital role in saving human lives, especially in emergencies such as accidents, surgeries, and serious medical conditions. In Pakistan, the current blood donation process mainly depends on personal contacts, hospital visits, and social media posts, which are slow and often unreliable. Blood banks often maintain records manually or with basic systems, making it hard to find updated donor information quickly. Because of this, patients and hospitals face delays in finding the right donors during critical moments. At the same time, many people want to donate blood, but they do not know where or when someone near them needs it.

With the increasing use of smartphones and mobile internet, a digital platform can easily solve these problems. A mobile application with real-time push notifications can quickly connect donors and patients in one place. The Life-Link Smart Blood Donation System is developed to provide a centralized and easy-to-use platform that makes blood donation faster, more efficient, and reliable across Pakistan.

1.2 Problem Statement

In Pakistan, it is still difficult to find blood quickly during emergencies. Patients who need blood due to accidents, surgeries, or childbirth often face problems in finding suitable donors on time. Most people depend on phone calls, social media posts, and personal contacts, which are slow and not always reliable. In many cases, blood banks and hospitals manage records manually, which leads to outdated or incorrect donor information.

There is no proper centralized system that connects donors and patients in one place. Because of this, communication is slow, especially during emergency situations. In addition, existing methods do not automatically verify donor eligibility (such as minimum age and donation cooldown

periods), which can cause safety issues and waste valuable time. Most existing apps also lack support for local languages like Urdu, making them difficult to use for many people. These problems cause serious delays and health risks. Therefore, there is a clear need for a simple, reliable, secure, and easy-to-use digital system to manage blood donation activities.

1.3 Problem Solution

The Life-Link Smart Blood Donation System is a software solution designed to solve problems in blood donation coordination and management. The system provides a centralized digital platform that directly connects donors and patients.

The mobile application allows users to register, request blood, and respond to donation requests easily. The system checks donor eligibility by verifying the user's age (18+ years) and the mandatory 90-day cooldown period since their last donation, along with a quick pre-donation health and weight checklist. It recommends potential donors based on blood group compatibility, user location, and donation history. Real-time push notifications are used to alert nearby donors during emergencies. The system also keeps user data secure and provides full support for both Urdu and English languages (including right-to-left layout) to make it accessible to everyone. By combining these features, Life-Link reduces coordination delays, improves communication, and helps patients find blood faster during medical emergencies.

1.4 Objectives of the Proposed System

- 1) To develop a cross-platform mobile application for donors and patients with bilingual support (English and Urdu).
- 2) To allow users to create and manage blood requests with different urgency levels (Normal, Urgent, and Critical).
- 3) To recommend and alert compatible, nearby donors using real-time push notifications and location filtering.
- 4) To verify donor eligibility through automated age and cooldown checks, supported by a pre-donation health checklist.
- 5) To facilitate direct communication via in-app chat, track donation history, and provide an admin dashboard for system management.

1.5 Scope

This project focuses on developing a centralized digital system for blood donation and management. The system includes a cross-platform mobile application for donors and patients, along with a web admin dashboard for administrators. Donors can register, manage their profiles, respond to blood requests, chat with requesters, and track their donation history. Patients can create blood requests with different urgency levels (Normal, Urgent, and Critical). The mobile app also provides bilingual support in English and Urdu.

The project includes a secure backend to verify donor eligibility and match nearby compatible donors based on blood group, location, and donation cooldown rules. Physical blood collection, laboratory testing, payment services, and direct hospital database integrations are not included in this project. The system focuses on core digital features like user management, blood requests, 1-on-1 chat, and real-time emergency notifications within the scope of a final year project.

1.6 System Components

The proposed system is divided into multiple modules based on their functionality and platform. Each module is designed to perform specific tasks required for the smooth working of the system.

1.6.1 Module 1: Client Mobile Application (Donor and Patient)

- User registration and login
- Profile management
- Blood request creation
- Search and Filter request
- View donation history
- Receive notifications
- Language support (Urdu and English)
- In-app chat

1.6.2 Module 2: Admin Web Application

- Admin login
- Manage users
- View blood requests and donations

- Basic reporting

1.6.3 Module 3: Backend System

- User authentication
- Data storage and retrieval
- API support for mobile and web apps
- Push notification service

1.7 Related System Analysis/Literature Review

Several blood donation apps and systems already exist, but they have limitations that reduce their usefulness in Pakistan. Most existing systems only provide simple donor lists and manual search, with limited real time updates. In addition, many apps have old and confusing designs that make them hard to use during emergencies. They also lack automatic donor matching, eligibility checks, direct chat between users, and Urdu language support. The proposed system solves these problems by providing a modern, simple, and easy to use solution designed specifically for local users.

Table 1.1 Related System Analysis with Proposed Project Solution

Application Name	Weakness	Proposed Project Solution
E-Blood	Limited real-time coordination and no proper donor-patient communication.	Provides real-time notifications and direct communication between donors and patients to improve response time.
Blood Help	Android-only application with limited features for managing donation requests.	Provides cross-platform mobile application with better request management for donors and patients.
Red Cross Blood Donor App	Not localized for Pakistan and does not support Urdu language.	Includes localized features with bilingual support and improved local coordination.
Blood Connect	Web-based only, with no mobile-first design and limited coordination features.	Offers a mobile-based system with improved coordination between donors, patients, and admin.

1.8 Vision Statement

For patients, and blood donors in Pakistan who need a fast, reliable, and easy way to manage blood donation during emergency and routine situations. The Life Link Smart Blood Donation System is a mobile and web-based system that helps connect donors and patients through a single digital platform using simple rule-based donor matching and real-time notifications. Unlike traditional manual methods and existing blood donation applications with limited coordination, our system provides a centralized, user-friendly, and locally suitable solution with bilingual support and improved communication and reducing delays in emergencies.

1.9 System Limitations and Constraints

This section describes the limitations and constraints of the proposed system. Limitations refer to external factors that are beyond the direct control of the development team, while constraints are internal factors defined by project scope, time, and available resources.

1.9.1 System Limitations

- The system depends on stable internet connectivity; limited or poor network access may affect real-time features such as notifications and data synchronization.
- The accuracy of donor eligibility and matching depends on the correctness of user-provided information.
- The system relies on third-party services for authentication and notifications, which may face temporary downtime or service limitations.
- The application does not include medical testing or blood quality verification, as these processes are handled by hospitals and blood banks.

1.9.2 System Constraints

- The scope of the project is limited to software development and does not include physical blood collection or hospital operations.
- Development is constrained by the final year project timeline approved by the university.

- The system is developed using selected technologies based on team expertise and available resources.
- The project focuses on core features to ensure stability and usability within the given constraints.

1.10 Tools and Technologies

Table 1.2 Tools and Technologies for Proposed Project

Tools	Version	Rationale
Visual Studio Code	Latest	Used for writing and managing frontend and backend source code
Android Studio	Latest	Used for Android builds
Xcode	16.4	Used for iOS build, and simulator support
Postman	Latest	Used for testing and validating backend APIs
GitHub	Cloud-based	Used for source code management and team collaboration
Technology	Version	Rationale
React Native	0.82	Used for cross-platform mobile application development
Next.js	16	Used for developing the web-based admin dashboard
NestJS	latest	Used for backend API development
Supabase	Cloud Service	Used for authentication, database, and real-time data handling
OneSignal	Cloud Service	Used for push notification services
AntiGravity	latest	Use for IDE and using ai agents to help code

1.11 Project Deliverables

The following are the main deliverables of the proposed project. All documentation and supporting material are included within the Final Year Project report as appendices.

1.11.1 Deliverables after 7th Semester

- Project planning and proposal documentation

- Problem statement, background study, and literature review
- Software Requirements Specification (SRS)
- System architecture and overall design
- UML diagrams (use case, class, sequence, activity)
- Database design and system flow diagrams

1.11.2 Deliverables after 8th Semester

- Functional mobile application implementing the proposed system.
- Web-based admin panel for management and monitoring.
- Backend services and database integration.
- Final Year Project report with complete documentation and implementation.

1.12 Project Planning

Gantt Chart representing the plans of carrying out the project activities, representing the milestones as well as resources required for each task

Task	Start Week	Duration (Weeks)	End Week	Milestone	Source / Resources
Topic Selection & Proposal Approval	1	1	1	Topic and proposal approved	Supervisor
Problem Study & Literature Review	2	2	3	Problem and existing work understood	Research papers, internet
Requirement Analysis & Scope Definition	3	2	4	Requirements finalized	Supervisor discussion
System & Database Design	5	2	6	Architecture and database designed	Design tools
UML Diagrams & Documentation	6	2	7	All diagrams prepared	PlantUML, Mermaid

Final Review & Submission	8	1	8	Project documents submitted	Supervisor review
---------------------------	---	---	---	-----------------------------	-------------------

Table 1.3 Week-wise Project Task Schedule and Milestones

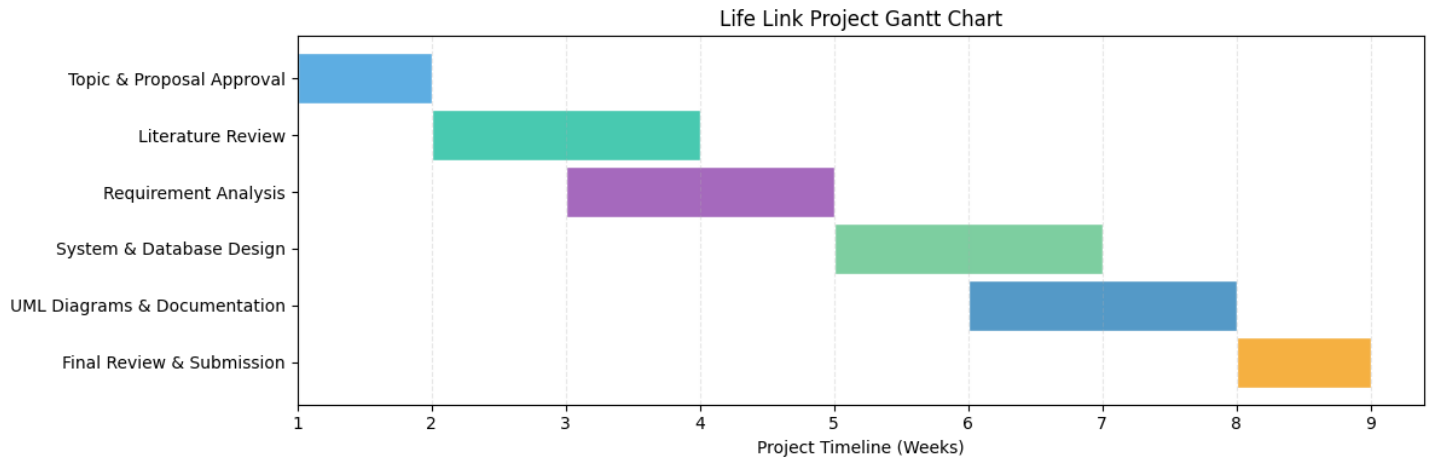


Figure 1 Gantt Chart

1.13 Summary

This chapter introduced the Life Link Smart Blood Donation System, explained the blood donation problems in Pakistan and described how the system connects donors and patients through one simple application. It also briefly covered the project objectives, scope, main modules, tools, limitations, and overall development plan to provide a foundation for the rest of the report.

Chapter 2

Requirements Analysis

2. Analysis

This chapter details the analysis of the Life Link Smart Blood Donation System, translating the identified problem into structured software requirements. It defines the system's users, outlines the process for gathering their needs, and specifies the functional and non-functional requirements. The analysis concludes with detailed use cases that describe the core interactions between users and the system, forming a clear blueprint for the design and development phases.

2.1 User classes and characteristics

The Life Link system serves multiple user groups, each with distinct needs, characteristics, and interaction patterns. Understanding these user classes is essential for designing appropriate functionalities and user experiences.

Table2.1: User Classes and Characteristics

User Class	User Characteristics
Mobile App Users (Donors & Patients)	Regular mobile users in Pakistan. A user can act as a Donor (view urgent requests, check if they can donate, respond to requests, and see their past donations) and as a Patient (post blood requests, see donor responses, and talk through in-app chat). Needs a simple, easy-to-use app with Urdu and English support and fast alerts.
System Admin	Admin users who log into the web dashboard to manage user accounts, check blood requests, view reports, and keep the system safe.

2.2 Requirement Identifying Technique

To derive a complete set of functional requirements, two complementary techniques were employed: Use Case Analysis as the primary method, supplemented by Informal Brainstorming.

2.2.1 Use Case Analysis

This technique was selected because the Life Link system is an interactive, multi-actor application. It effectively captures the specific goals and step-by-step interactions of each user class (Donor, Patient and Admin) with the system.

- UC1: Register/Login User

- UC2: Create Blood Request
- UC3: Check Donor Eligibility
- UC4: Fulfil Blood Request
- UC5: Chat with Requester/Donor
- UC6: View System Reports (Admin)
- UC7: Send/Receive Notifications
- UC8: Update User Profile
- UC9: Manage Users (Admin)
- UC10: View Donation History
- UC11: Search/Filter Blood Requests

2.2.2 Brainstorming

Brainstorming was used as a supporting technique to identify additional functional requirements. Basic discussions within the project team helped in refining system features based on logical analysis of the problem domain and expected user interactions. This approach assisted in validating use cases, identifying essential features, and ensuring that the requirements remained simple, relevant, and aligned with the project objectives.

2.3 Functional Requirements

This section details the functional requirements of the Life Link system, specifying the software capabilities that must be implemented to enable the services identified in the use cases. The requirements are organized by core system features and are written from the user's perspective. Each requirement is uniquely labeled, and its attributes such as rationale, source, and priority are defined to ensure clear traceability and implementation guidance.

2.3.1 Functional Requirement 1: User Registration and Authentication

Table2.2: Description of FR-1

Identifier	FR-1
Title	User Registration and Login
Requirement	: A user shall be able to register for a new account or log into an existing account using an email address and password, or via Google Sign-In, within

	2 minutes.
Source	Use Case UC1: Register/Login User
Rationale	Secure and controlled access is the foundation for all personalized system features and data privacy.
Business Rule (if required)	Each email address must be unique to a single account. Passwords must be at least 6 characters long and include uppercase, lowercase, numbers, and special characters.
Dependencies	None (Foundational)
Priority	High

2.3.2 Functional Requirement 2: Blood Request Creation

Table2.3: Description of FR-2

Identifier	FR-2
Title	Create Blood Request
Requirement	A user shall be able to create a new blood request by specifying the required blood group, number of units needed, hospital name, city, location coordinates, and urgency level (Normal, Urgent, or Critical).
Source	Use Case UC2: Create Blood Request
Rationale	Allows patients or their families to formally state their need, which initiates the donor matching and push notification process.
Business Rule (if required)	Blood group must be selected from the standard predefined list (A+, A-, B+, B-, AB+, AB-, O+, O-). The quantity must be a positive whole number (minimum 1 unit). Hospital name must be at least 3 characters.
Dependencies	FR-1 (User must be logged in with a completed profile)
Priority	High

2.3.3 Functional Requirement 3: Donor Eligibility Check

Table2.4: Description of FR-3

Identifier	FR-3
Title	Check Donor Eligibility
Requirement	The system shall automatically check a donor's eligibility by evaluating their age (18+ years) and the mandatory 90-day cooldown period since their last donation, displaying their status (Eligible to Donate or Cooldown Period with Next Eligible Date). When responding to a request, a pre-donation health checklist shall verify that the donor weighs at least 50 kg and is in good health before confirming the donation.
Source	Use Case UC3: Check Donor Eligibility
Rationale	Ensures donor safety and blood recipient safety by preventing ineligible, underage, or unwell individuals from donating
Business Rule (if required)	Minimum donor age is 18 years. A minimum gap of 90 days is required between whole blood donations. Donors must confirm weight ≥ 50 kg and no active fever or illness prior to donation commitment.
Dependencies	FR-1, FR-8 (Profile data and donation records)
Priority	High

2.3.4 Functional Requirement 4: Donor Matching and Notifications

Table2.5: Description of FR-4

Identifier	FR-4
Title	Match Donors and Send Notifications
Requirement	Upon creation of a new blood request, the system shall identify suitable donors based on blood group compatibility, city/location, and eligibility

	status. It shall then send real-time push notifications to matching eligible donors who have notifications enabled.
Source	Use Case UC2: Create blood request & UC7: Send/Receive Notifications
Rationale	Minimize the time between a request being posted and a suitable nearby donor being alerted.
Business Rule (if required)	Notifications are sent to eligible donors with compatible blood groups. Urgent and Critical requests prioritize donors within the same city or geographic area.
Dependencies	FR-2, FR-3
Priority	High

2.3.5 Functional Requirement 5: Blood Donation Fulfillment

Table2.6: Description of FR-5

Identifier	FR-5
Title	Fulfill a Blood Request
Requirement	A donor shall be able to accept a blood request, confirming their intent to donate. Upon completion of the donation, the request status shall be updated to 'Fulfilled' (or Partially Fulfilled if more units are needed), and the donation shall be recorded in the donor's history.
Source	Use Case UC4: Respond and Fulfil Blood Request
Rationale	Closes the lifecycle of a blood request and maintains accurate records of completed donations for both patients and donors.
Business Rule (if required)	Once marked fulfilled, the donor's last donation date is updated automatically, initiating the 90-day cooldown period and updating the lives saved counter.
Dependencies	FR-2, FR-3, FR-4
Priority	High

2.3.6 Functional Requirement 6: Donation History and Tracking

Table2.7: Description of FR-6

Identifier	FR-6
Title	View Donation History and Stats
Requirement	A donor shall be able to view a list of all their past donations, including dates, hospital locations, total donations count, estimated lives saved, and their current eligibility cooldown status.
Source	Use Case UC10: View Donation History and Stats
Rationale	Provides donors with a record of their contributions, calculates lives saved, and helps them track when they are eligible to donate again.
Business Rule (if required)	History and statistics are generated automatically from fulfilled donations. Donors can only view their own history.
Dependencies	FR-5
Priority	Medium

2.3.7 Functional Requirement 7: Bilingual User Interface

Table2.8: Description of FR-7

Identifier	FR-7
Title	Bilingual Interface Support
Requirement	The user shall be able to switch the application interface language between English and Urdu at any time from settings. All static text, labels, menus, and layout directions (LTR and RTL) shall update accordingly.
Source	Comparative Analysis & Project Objective.
Rationale	Makes the application accessible, intuitive, and user-friendly for the wider population of Pakistan.

Business Rule (if required)	The language setting must be persistent per user account and applied across all mobile app screens upon launch.
Dependencies	None (Front-end implementation)
Priority	High

2.3.8 Functional Requirement 8: User Profile Management

Table2.9: Description of FR-8

Identifier	FR-8
Title	Update User Profile
Requirement	A logged-in user shall be able to view and edit their profile information, including full name, contact details, date of birth, blood group, city/location, notification preferences, and profile image.
Source	Use Case UC-8: Update User Profile
Rationale	To ensure user information remains accurate and up to date for effective communication and matching.
Business Rule (if required)	Updating Date of Birth re-validates the minimum 18-year age requirement. Updating city or location recalculates proximity matching for nearby requests.
Dependencies	FR-1 (User authentication)
Priority	Medium

2.3.9 Functional Requirement 9: Administrative User Management

Table 2.10: Description of FR-9

Identifier	FR-9
Title	Manage Users (Admin)

Requirement	The System Administrator shall be able to view a list of all users on the web dashboard, search users by name or email, view user statuses and requests, and suspend or activate accounts.
Source	Use Case UC-9: Manage Users and Requests (Admin)
Rationale	To maintain system integrity, prevent misuse or spam, and manage users effectively.
Business Rule (if required)	Administrative functions are restricted to verified administrator accounts with role-based access control.
Dependencies	FR-1 (Admin authentication)
Priority	Medium

2.4 Non-Functional Requirements

This section specifies nonfunctional requirements other than constraints, supporting requirements recorded in section 2.3, and external interface requirements. These quality requirements should be specific, quantitative, and verifiable. The following are some examples of documenting guidelines.

2.4.1 Reliability

- System shall maintain 99% uptime annually (excluding maintenance).
- Operational error rate shall not exceed 0.5% of transactions.
- System recovery after failure shall occur within $\leq$ 10 minutes.
- User & request data shall maintain $\geq$ 99% accuracy with immediate updates

2.4.2 Usability

- New users shall complete registration within $\leq$ 3 minutes.
- Users with basic mobile experience shall navigate the app within $\leq$ 5 minutes without help.

2.4.3 Performance

- System shall respond to login, blood requests, and data actions within $\leq$ 2 seconds for 90% of operations.
- Emergency blood request notifications shall be delivered to eligible donors within $\leq$ 3

seconds using Supabase Real-time + One Signal.

- System shall support 10,000 concurrent users without performance degradation

2.4.4 Security

- All users shall authenticate using Supabase Auth with JWT-based authentication.
- Stored data must use AES-256, and all communication shall use TLS 1.3.
- Admin-level functions only accessible to predefined, verified admins.

2.4.5 Maintainability

- 80% + of the codebase shall be modular and component-based.
- Critical issues must be diagnosable and fixable within ≤ 24 hours using Sentry.
- All major modules shall maintain $\geq 90\%$ documentation completeness.

2.4.6 Portability

- App shall support Android 8.0+ and iOS 12+
- Installation shall complete within ≤ 120 seconds on a 4G network.
- App size (APK/IPA) shall not exceed 150 MB.

2.5 External Interface Requirements

This section provides information to ensure that the system will communicate properly with users and with external hardware or software elements.

2.5.1 User Interfaces Requirements

- The mobile app shall support a minimum screen resolution of 720×1280 px.
- Navigation shall follow standard mobile patterns with consistent placement of home, back, and help buttons.
- The system shall provide visual feedback for user actions, such as button clicks and form submissions.
- Optional biometric authentication (fingerprint, Face ID) may be supported for login using device-native APIs.

Note: Specific screen layouts and mock-ups are provided for reference, but the final UI may change during development. Storyboards included in the SRS are illustrative and not binding designs.

2.5.2 Software interfaces

- The system backend shall be accessible via RESTful APIs.
- The system shall interact with the Supabase database for real-time data synchronization and storage.
- APIs shall be compatible with standard HTTP methods and JSON data format.
- The architecture allows future migration to other hosting platforms or cloud providers without impacting functionality

2.5.3 Hardware interfaces

- The system is designed for mobile devices running Android 8.0+ and iOS 12+.
- *Mobile device sensors (GPS/Location services for proximity calculations) are utilized by the application.*
- No additional specialized hardware is required for the basic system functionalities

2.5.4 Communications interfaces

- The system shall use push notifications for alerts using One Signal for mobile devices.
- Real-time updates for donor-patient interactions shall be handled via Supabase Real-time.
- Standard network protocols (HTTPS) shall be used for all client-server communication.
- The system shall provide error messages and feedback for failed network request

2.6 Use case Analysis

2.6.1 Use Case 1: Register/Login User

Table2.11: Description of Use-case 1

UC Identifier	UC-1
UC Name	Register/Login User
Actors	Donor, Patient
Description	Allows a new user to create an account or an existing user to authenticate and

	access the system.
Trigger	User opens the mobile application for the first time or returns to use it.
Preconditions	• Device has an active internet connection.
Post conditions	• User account is created or authenticated • User is redirected to the home screen. • A valid session token is stored.
Normal Flow	• User opens the Life Link app. • User selects Register. • User enters basic details and submits the form. • System validates data and creates the account. • Verification email is sent to the user. • After verification, the user is logged in and redirected to the home screen. • After registration/login, if the user hasn't completed their profile (is_onboarded === false), the app directs them to the Complete Profile (Onboarding) screen to set their blood group, DOB, and city before reaching the Home Screen.
Alternate Flows	AF1 - Login: If the user selects "Login", they enter their email and password; the system authenticates them and redirects to the dashboard. AF2 - Social Login: User chooses "Login with Google"; the system handles OAuth authentication via Supabase and creates/logs in the user. AF3 - Forgot Password: User selects "Forgot Password", enters email, receives a reset link, and sets a new password.
Business Rules	• Each email address can be associated with only one account. • Passwords must be at least 6 characters long and include one special character.
Assumptions	Users have a valid email address and can access it for verification.

2.6.2 Use Case 2: Blood Request Creation

Table2.12: Description of Use-case 2

UC Identifier	UC-2
---------------	------

UC Name	Create Blood Request
Actors	Patient, System
Description	Allows a patient user to post a formal request for blood, initiating the matching process.
Trigger	A patient is in need of blood and navigates to the "Request Blood" section of the app.
Preconditions	• The patient user is logged in. • The patient's profile is complete (at least blood group and location). • Internet connection is available.
Post conditions	• A new blood request is saved in the system database. • Notifications are sent to eligible, nearby donors. • The request appears in the patient's request list and in the public feed for matching donors.
Normal Flow	• Patient opens Request Blood screen. • Selects blood type and units needed. • Enters hospital name and confirms location and urgency level (Normal, High, Critical). • Submits request. • System validates data, saves the request, and displays confirmation with request ID.
Alternate Flows	AF1- High/Critical Request: If marked "High" or "Critical", the system visually prioritizes the request on the feed and home screen banner.
Business Rules	• Blood type must be selected from the valid list: A+, A-, B+, B-, O+, O-, AB+ and AB-. • The number of units must be a positive integer (typically 1-10). • A patient cannot have more than 3 active "Normal" requests at a time
Assumptions	The patient has the necessary information (blood type, hospital location) to complete the request.

2.6.3 Use Case 3: Check Donor Eligibility

Table2.13: Description of Use-case 3

UC Identifier	UC-3
UC Name	Check Donor Eligibility
Actors	Donor, System
Description	The system evaluates a donor's eligibility to give blood based on their profile data and predefined medical rules, providing a clear status
Trigger	A donor manually requests an eligibility check, or the system automatically checks during the matching process for a new request
Preconditions	• Donor is logged in with a completed profile. • System is operational and has access to donor's past donation records.
Post conditions	• Eligibility status ("Eligible", "Not Eligible", or "Next Eligible Date: [date]") is displayed to the donor. • The donor's "availability status" in the system is updated based on the result. • The check is logged for record-keeping.
Normal Flow	• Donor opens the Donation Dashboard / My Donations screen. • System automatically evaluates age (18+) and the 90-day cooldown period from previous donations. • System displays a clear status. • When responding to a blood request, the system presents an interactive Health Checklist modal to confirm weight (≥ 50 kg) and current health readiness before finalizing the commitment.
Alternate Flows	• AF1 - Incomplete Profile: If profile is incomplete system navigate users to onboarding screen. • AF2 - Temporarily Ineligible: If ineligible due to recent donation, the system calculates and displays the next eligible date and offers to set a reminder.
Business Rules	• Eligibility rules are based on the World Health Organization (WHO) and Pakistan Red Crescent Society guidelines.
Assumptions	The donor provides accurate and truthful health information. The system provides a reliable assessment based on the given inputs.

2.6.4 Use Case 4: Fulfill Blood Request

Table2.14: Description of Use-case 4

UC Identifier	UC-4
UC Name	Fulfill Blood Request
Actors	Donor, System
Description	“Allows a matched donor to accept a blood request and confirm donation, updating all relevant records.
Trigger	A donor accepts a notification for a matched blood request.
Preconditions	● Actor is logged in (as Donor). ● A matching active blood request exists. ● For donors: They must be currently eligible (UC-3).
Post conditions	● The blood request status is updated (to "Matched" or "Fulfilled"). ● Donor's donation history is updated. ● Donor's "last donation date" is reset. ● Patient receives a notification about the status change.
Normal Flow	● Donor opens “Help Now” and selects a request. ● Taps “I Can Donate”. ● System checks eligibility and notifies patient. ● Donor donates blood. ● System updates records and thanks donor.
Alternate Flows	AF1 - Donor Cancels: After matching, if the donor cannot proceed, they can cancel; the request returns to the active pool for other donors.
Business Rules	● Only one donor can be primarily matched to a single request unit at a time to avoid confusion. ● A request can be partially fulfilled by multiple donors if multiple units are needed.
Assumptions	The donation process is completed physically at a medical facility and system records the confirmation.

2.6.5 Use Case 5: Chat with Requester/Donor

Table2.15: Description of Use-case 5

UC Identifier	UC-5
UC Name	Chat with Requester/Donor
Actors	Donor, Patient
Description	Allows a donor and patient to communicate directly through real-time in-app messaging once a donation response is initiated.
Trigger	A donor responds to a blood request, or a user opens an ongoing conversation from the Chats tab.
Preconditions	• Both users are logged in with active accounts. • An active blood request thread exists between the two users. • Internet connection is available.
Post conditions	Messages are delivered in real time and stored securely in the database. • Unread message count badges are updated for the recipient.
Normal Flow	• Donor taps "Donate Blood" on a request card. • System opens the private 1-on-1 chat thread between the donor and requester. • Users exchange messages, discuss hospital location, and confirm timing. • System delivers messages instantly with timestamps.
Alternate Flows	AF1 - Offline Recipient: If the recipient is offline, the message is stored on the server and delivered as a push notification alert.
Business Rules	• Chat conversations are private and only accessible to the participating donor and requester.
Assumptions	Both users use the chat respectfully to coordinate blood donation logistics.

2.6.6 Use Case 6: View System Reports

Table2.16: Description of Use-case 6

UC Identifier	UC-6
UC Name	View System Reports

Actors	System Administrator
Description	Provides administrators with analytical dashboards and detailed reports on system usage, donation metrics and user activity.
Trigger	An administrator logs into the web dashboard and navigates to the reporting section to monitor performance or prepare a summary.
Preconditions	• User is logged in with administrative privileges. • The system has collected operational data over a period of time
Post conditions	• The requested reports are generated and displayed. • Data can be exported or shared if needed.
Normal Flow	• Admin logs into the dashboard. • Opens Reports & Analytics. • Selects report type and filters. • System generates charts and tables. • Admin reviews the report.
Alternate Flows	AF1 - Export Report: Administrator clicks "Export", selects format (PDF/Excel), and the system generates a downloadable file. AF2 - Schedule Report: Administrator can schedule a specific report to be emailed weekly/monthly to designated recipients.
Business Rules	• Report data must be secure and accessible only to administrators. • All report data must reflect real-time or near real-time information. • Personally information of donors or patients is anonymized in aggregate reports.
Assumptions	The underlying database contains sufficient historical data to generate meaningful reports.

2.6.7 Use Case 7: Send / Receive Notifications

Table2.17: Description of Use-case 7

UC Identifier	UC-7
UC Name	Send /Receive Notifications
Actors	System, Donor, Patient

Description	The system automatically sends timely, relevant push notifications to users' mobile devices for critical events, and users receive and interact with these alerts.
Trigger	A system event occurs that requires user awareness or action (new blood request, donation pledge, or new chat message).
Preconditions	• Users have allowed notification in permissions on their mobile device. • The notification service (One Signal) and internet are operational. • For targeted notifications, the user must meet specific criteria (location, eligibility, etc.).
Post conditions	• Notification is delivered to the user's device. • User can tap the notification to open the app to the relevant screen. • The system logs the delivery and interaction.
Normal Flow	• System triggers an emergency event (e.g., critical blood request created). • Eligible nearby donors with matching blood groups are identified. • System sends push notification via OneSignal API with key details. • Donor receives and taps the notification alert. • Mobile app opens the request details screen directly.
Alternate Flows	AF1 - User Manages Preferences: User goes to Settings to toggle notification preferences.
Business Rules	• Emergency request notifications must be sent within 3 seconds of request creation. • Users can disable non-critical notifications from their profile settings.
Assumptions	Users keep push notifications enabled on their devices.

2.6.8 Use Case 8: Update User Profile

Table2.18: Description of Use-case 8

UC Identifier	UC-8
UC Name	Update User Profile
Actors	Donor, Patient

Description	Allows users to view and modify their personal information, contact details, health information (for donors), and preferences stored in their account.
Trigger	User needs to correct information, change contact details, or update health status (e.g., after a donation or medical event).
Preconditions	• User is logged in. • User has a valid, active account.
Post conditions	• User profile is updated in the database with new information. • For donors, if health data changed, eligibility is automatically re-evaluated (UC-3). • A confirmation message is displayed to the user.
Normal Flow	• User opens Profile and taps Edit. • System shows editable profile details. • User updates information and taps Save. • System validates and saves changes. • Eligibility is re-checked if health data changed. • System shows Profile Updated Successfully.
Alternate Flows	AF1 - Change Password: User selects "Security" and follows steps to change password with old password verification.
Business Rules	• Certain fields (like User ID, account creation date, email) are read-only.
Assumptions	Users update their information truthfully and in a timely manner.

2.6.9 Use Case 9: Manage Users (Admin)

Table2.19: Description of Use-case 9

UC Identifier	UC-9
UC Name	Manage Users (Admin)
Actors	System Administrator
Description	Allows system administrators to oversee all user accounts, perform verifications, manage roles, and suspend or delete accounts if necessary to maintain platform integrity.
Trigger	An administrator logs into the admin dashboard to review user accounts,

	respond to a report, or perform routine maintenance.
Preconditions	Administrator is logged into the web admin panel
Post conditions	- User account status or details are updated as per the admin action. - The action is recorded in the admin audit log.
Normal Flow	- Admin opens User Management. - System shows a searchable user list. - Admin selects a user and views details. - Admin chooses an action (verify, suspend, warn, delete). - System asks for confirmation for critical actions. - System applies the change and logs the action.
Alternate Flows	AF1 - Bulk Actions: Administrator can select multiple users and apply the same action to all. AF2 - Restore Account: For a suspended user, administrator can choose "Reactivate" to restore access.
Business Rules	- An administrator cannot delete or modify the role of another administrator. - A user account can be suspended for violations of terms of service; data is retained for a defined period.
Assumptions	Administrators act responsibly and in accordance with platform policies and data protection regulations.

2.6.10 Use Case 10: View Donation History

Table2.20: Description of Use-case 10

UC Identifier	UC-10
UC Name	View Donation History
Actors	Donor
Description	Provides donors with a time-based record of all their past blood donations, including details of each event, for personal tracking and motivation.
Trigger	A donor wants to review their past donations.
Preconditions	- Donor is logged in. - The donor has made at least one donation recorded in the system (via

	UC-4).
Post conditions	• Donor views their historical donation list. • Donor may have the option to export or share this history.
Normal Flow	• Donor navigates their profile and selects "Donation History". • System fetch and displays a list of donations, sorted by date (newest first). • Each entry shows: Date, Donation Location, Blood Type Donated, Units Donated, and Associated Request ID (if any). • Donor can scroll through the list. • Donor can tap on any entry to see more details (e.g., recipient feedback if anonymous).
Alternate Flows	AF1 - No History: If the donor has no recorded donations, the system displays a motivational message and links to current requests. AF2 - Filter/Search: Donor can filter the list by date range using provided filter options.
Business Rules	• Donation history is generated automatically from fulfilled requests (UC-4). • History is private and accessible only by the donor themselves (and system admins for audit). • Each donation record is immutable once created.
Assumptions	All donations made through the platform are recorded accurately and completely.

2.6.11 Use Case 11: Search and Filter Blood Requests

Table2.21: Description of Use-case 11

UC Identifier	UC-11
UC Name	Search and Filter Blood Requests
Actors	Donor, Patient
Description	Allows users to search and filter active blood requests based on blood group, city, and urgency level.

Trigger	A user opens the Feed screen or taps Find Donor and Donate Blood from the Home screen.
Preconditions	● Device has an active internet connection. ● Active blood requests exist in the system database.
Post conditions	● A filtered list of matching blood requests is displayed on the screen.
Normal Flow	● User navigates to the Blood Requests Feed screen. ● User applies filters such as Blood Group (e.g., O+, B-), Urgency (Urgent, Critical), or City. ● System queries the database and updates the feed in real time. ● User views request cards showing patient name, hospital, units needed, distance, and urgency badge. ● User can tap any request card to view complete details and respond.
Alternate Flows	AF1 - No Matches Found: If no active requests match the selected filters, the system displays an empty state message suggesting the user reset filters.
Business Rules	● Fulfilled or cancelled requests are automatically hidden from the active public feed. ● Critical and Urgent requests are visually prioritized at the top of the feed.
Assumptions	Users regularly check the feed to find emergency requests in their area.

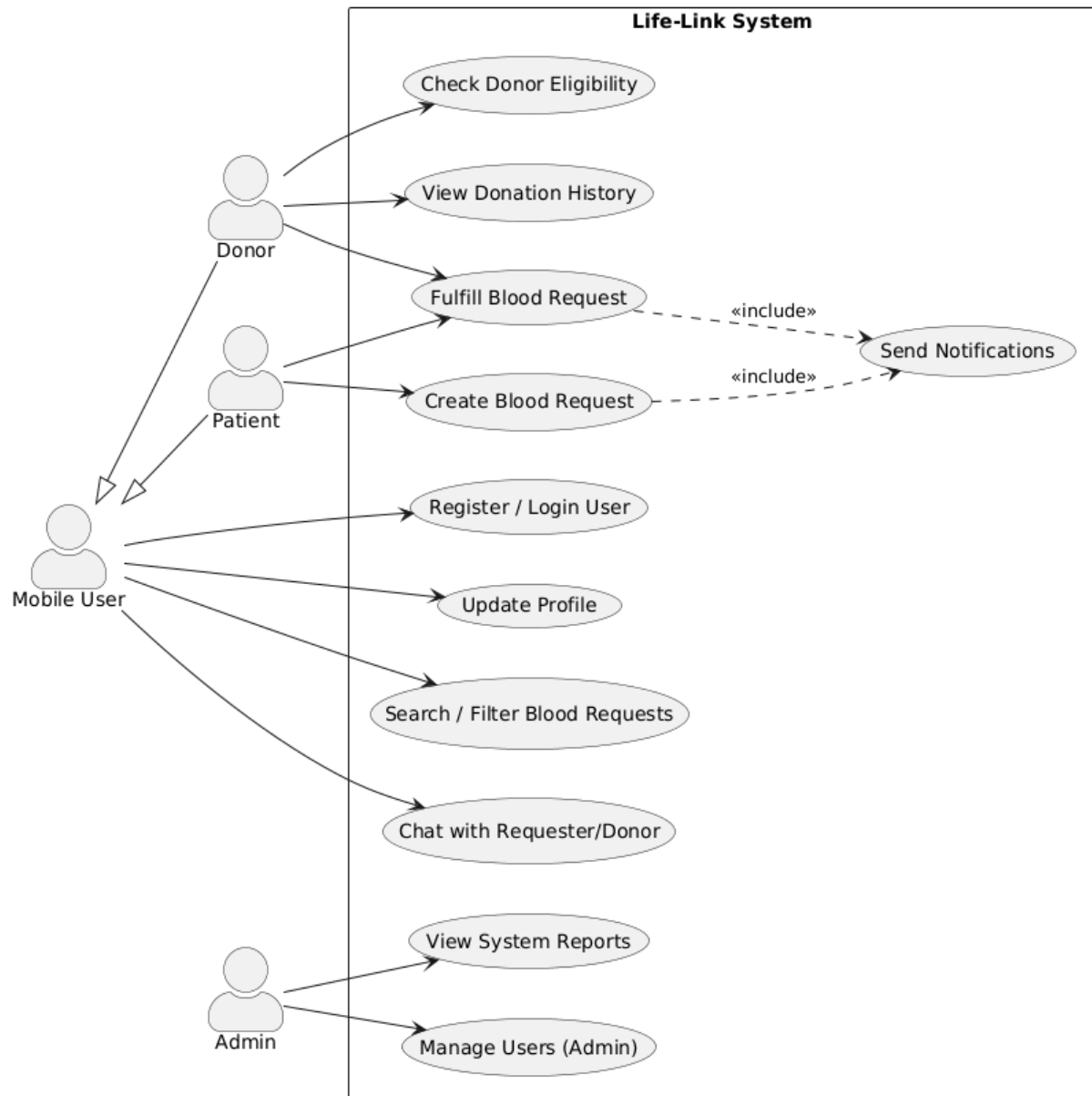


Figure 2 Use Case Diagram

2.7 Summary

This chapter explains how the requirements of the Life Link Smart Blood Donation System were identified and organized. It introduces the main user groups and the techniques used, such as brainstorming and use case analysis. The chapter summarizes the functional requirements like blood requests, donor matching, notifications, and admin control, as well as non-functional requirements such as performance, security, and usability. It also includes key use cases that show how users interact with the system.

Chapter 3

Design and Architecture

3. System Design

Explain the product perspective in which software will operate including dependencies and interaction with other system. List the design constraints (like performance requirements) impacting design.

3.1 Design considerations

This section defines the foundational constraints that shaped the architecture of the Life Link Smart Blood Donation System. It details the operational environment, core dependencies, and inherent limitations that directly influenced key design decisions to ensure the system is aligned with its objectives.

3.1.1 Assumptions and Dependencies:

The system design is based on the following critical assumptions and external dependencies:

Assumption:

- 1) **User Environment**
Users (donors and patients) have smartphones (Android 8.0+ or iOS 12+) and a stable internet connection for app usage and notifications.
- 2) **Data Accuracy**
Users provide correct and honest personal and medical information during registration and profile updates.
- 3) **User Participation**
Donors actively respond to blood requests and keep their availability status updated in the system.
- 4) **Service Availability**
Third-party services (Supabase for database/authentication and OneSignal for notifications) remain available and reliable as per their service agreements.
- 5) **System Logic**
The system uses simple rule-based eligibility checks which provide accurate results based on user-provided data.

Dependencies:

1. **Backend & Database (NestJS & Supabase):** The backend services, relational database (PostgreSQL), user authentication (JWT), and real-time listeners depend on NestJS and Supabase.
2. **Push Notifications (OneSignal):** Delivery of critical, time-sensitive emergency alerts to mobile devices relies on the OneSignal cloud service.
3. **Device Hardware & Permissions:** Location-based distance calculations require access to the device's GPS hardware and location permissions.
4. **Mobile Operating Systems:** The React Native mobile framework depends on the underlying Android (Google) and iOS (Apple) runtime environments.

3.1.2 Limitations:

The designed system acknowledges the following inherent limitations:

1. **Connectivity Dependency:** All real-time features (notifications and matching) require an active internet connection. Functionality is not usable in offline scenarios.
2. **Data Verification Boundary:** The system performs logical eligibility checks based on user-inputted data but cannot physically verify medical conditions, blood quality, or hemoglobin levels.
3. **Integration Scope:** The system is designed as a standalone coordination platform. It does not include direct electronic integration with existing Hospital Management Systems (HMS) or national health databases, which may limit automated data flow in some settings.
4. **System Scope:** The system uses simple rule-based logic for eligibility and does not include advanced prediction or analysis features.
5. **Geographic Precision:** Location services are subject to the accuracy of the device's GPS and network-based location, which may vary, especially in dense urban or remote rural areas.

3.1.3 Risks:

Potential risks were identified and handled through specific design choices:

1. Data Security & Privacy Breach:

- Loss of sensitive user health and contact data.
- Solution: Implemented Supabase Auth with JWT, enforced Row-Level Security (RLS) policies in the database.

2. System Performance Under Load:

- Slow response during emergencies leading to user abandonment.
- Solution: Adopted a scalable backend (NestJS) and a managed database (Supabase). Designed to support 10,000 concurrent users with key operations under 2-second response time.

3. Low User Adoption/Engagement

- The system may not be effective if users do not use it.
- Solution: Designed a bilingual (Urdu/English), intuitive UI and Incorporated gamification elements (donation history) to encourage repeat use.

4. Third-Party Service Failure:

- Core features (Auth, notifications) become unavailable.
- Implemented active network state monitoring (NetInfo), global Axios error interceptors with user-friendly retry prompts, and Sentry crash monitoring for rapid issue diagnosis.

3.2 Design Models

This section explains the main design models used to describe the internal structure and working of the Life Link Blood Donation System. The system is developed using an object-oriented approach, therefore UML diagrams are used to visualize different aspects of the system such as structure, interaction, and behavior.

The applicable models for the project using object-oriented development approach may include:

3.2.1 Class Diagram

This section explains the main design models used to describe the internal structure and working

of the Life Link Blood Donation System. The system is developed using an object-oriented approach, therefore UML diagrams are used to visualize different aspects of the system such as structure interaction, and behavior.

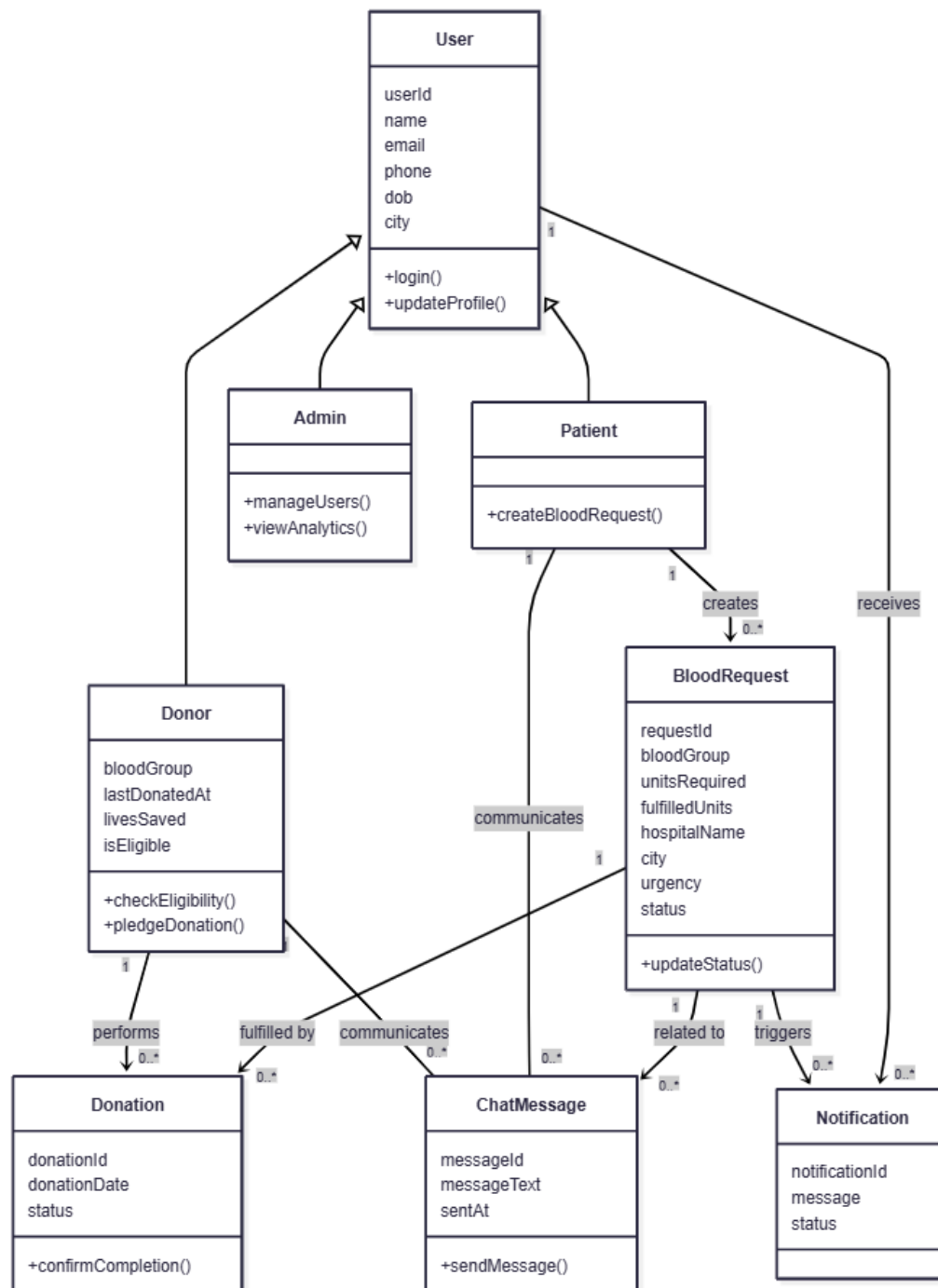


Figure 3 Class Diagram

3.2.2 Sequence Diagram

The interaction diagram describes how objects communicate with each other over time. It shows the flow of messages between system components in a specific order.

1. Auth Flow:

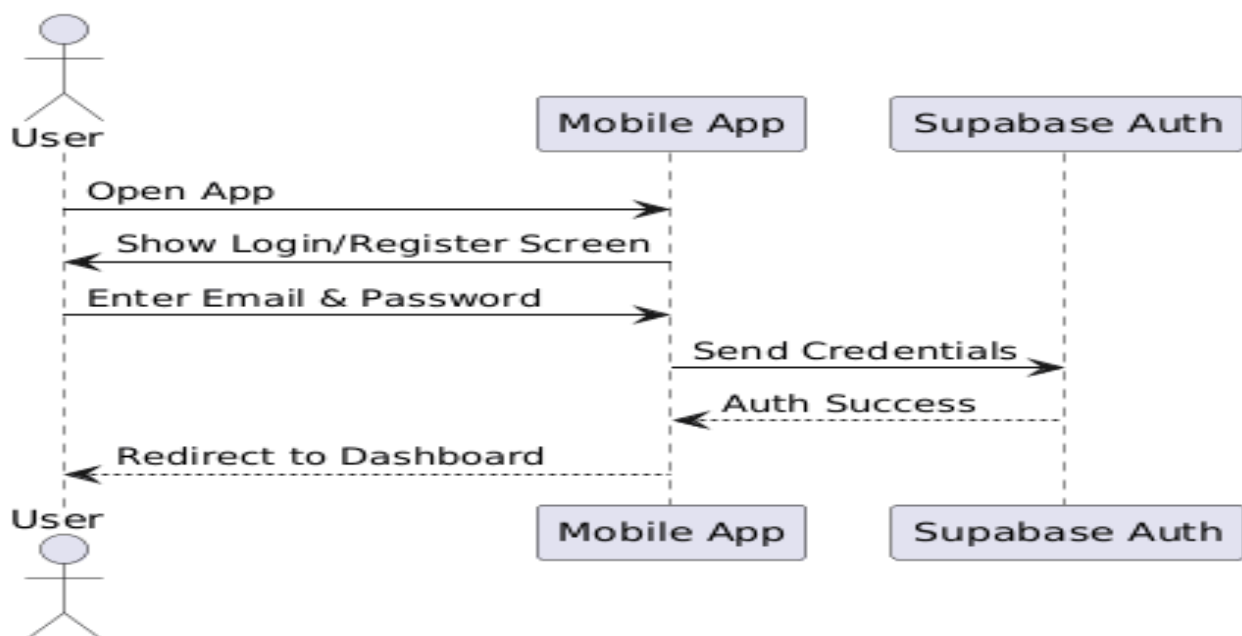


Figure 4 Sequence Diagram Auth Flow

2. Create Request Flow

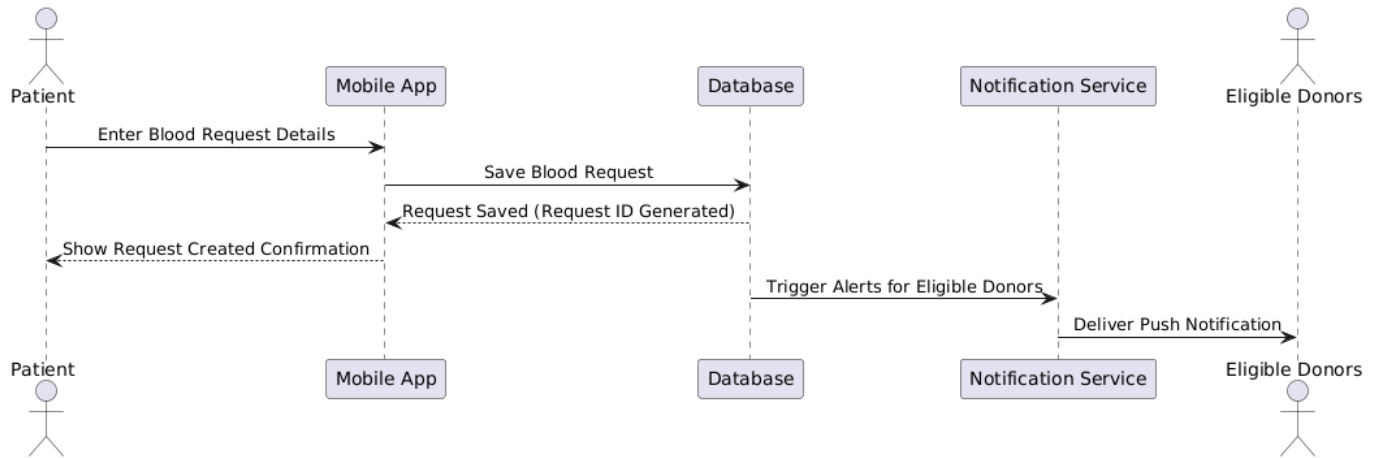


Figure 5 Sequence Diagram Create Blood Request

3. Check Eligibility Flow:

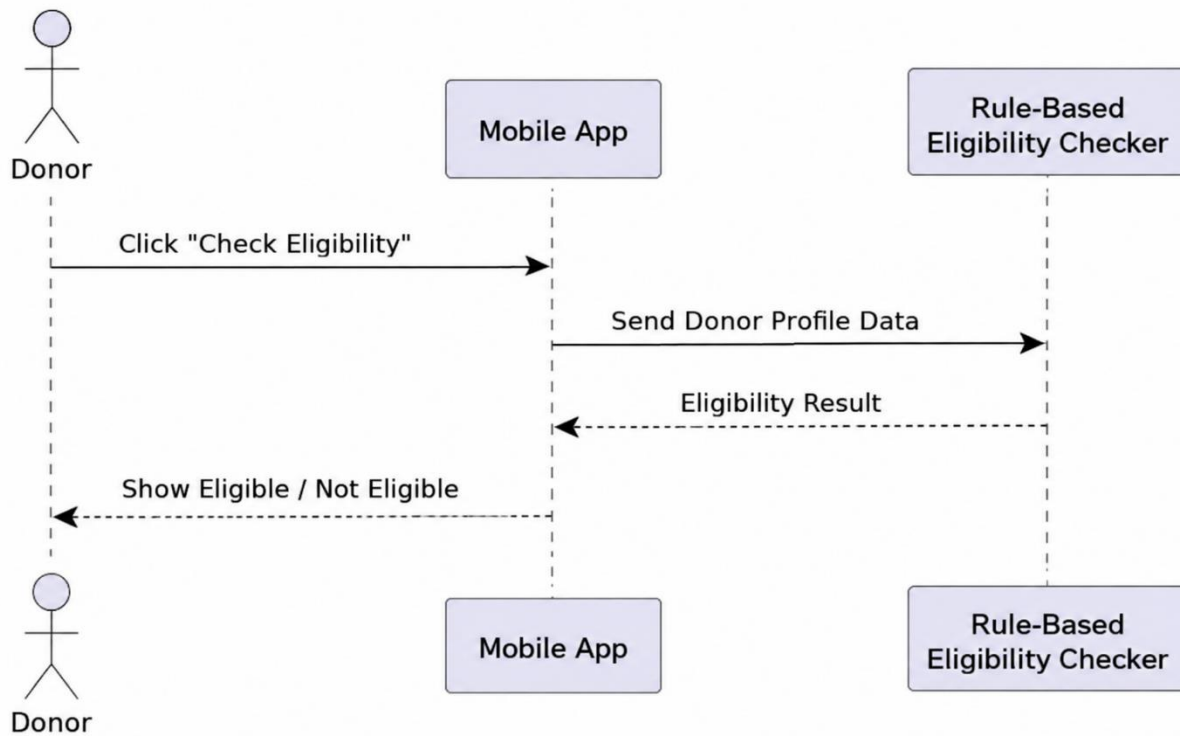


Figure 6 Sequence Diagram Check eligibility

4. Request fulfill flow:

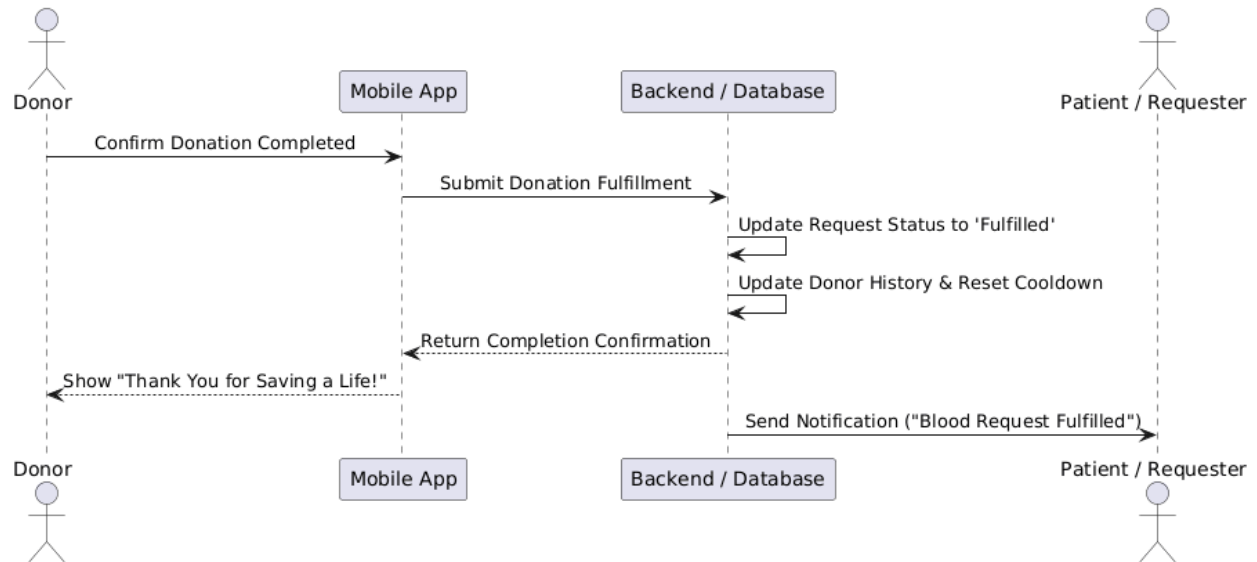


Figure 7 Sequence Diagram Request fulfill flow

5. Notification Flow:

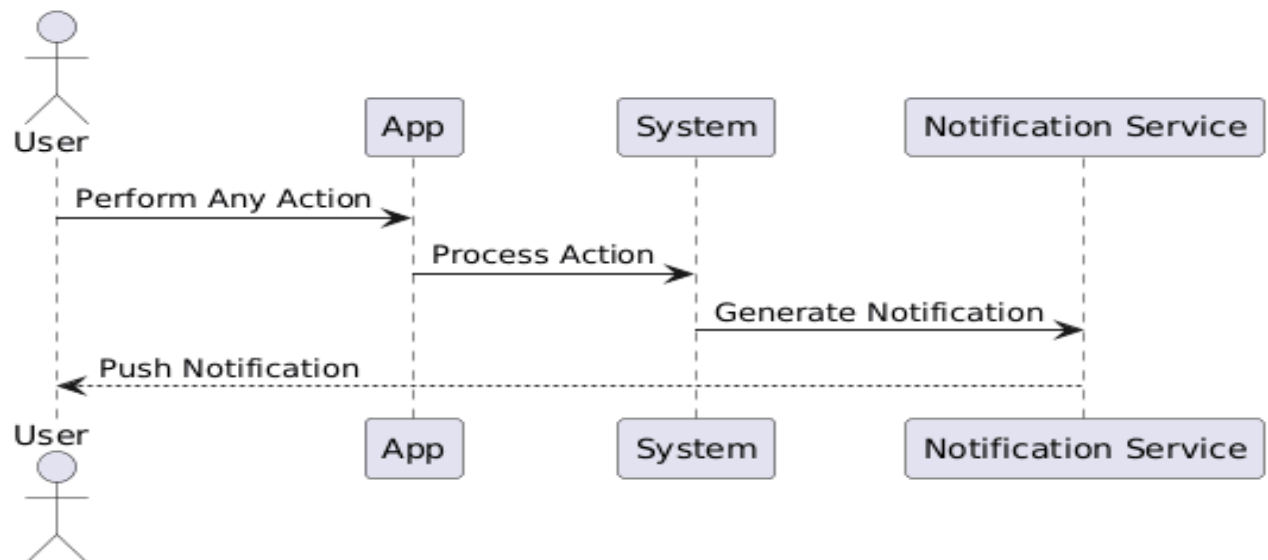


Figure 8 Sequence Diagram Notification flow

6. Chat flow

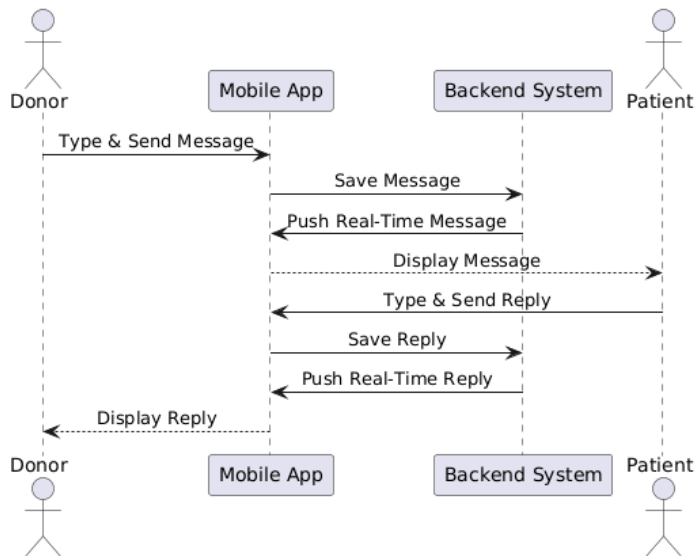


Figure 9 Sequence Diagram Chat flow

7. Update profile flow:

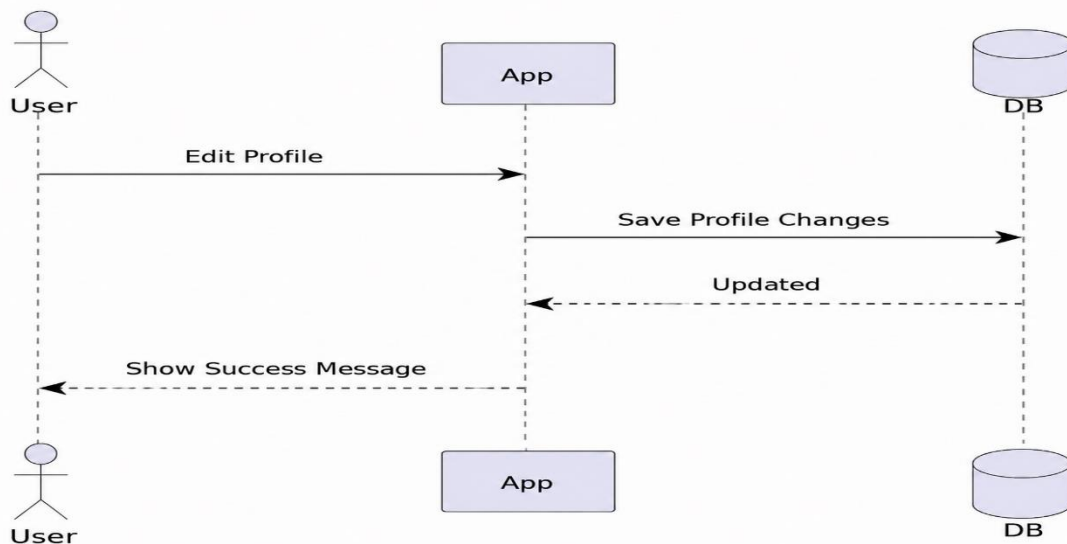


Figure 10 Sequence Diagram Update profile flow

8. Manage User Flow:

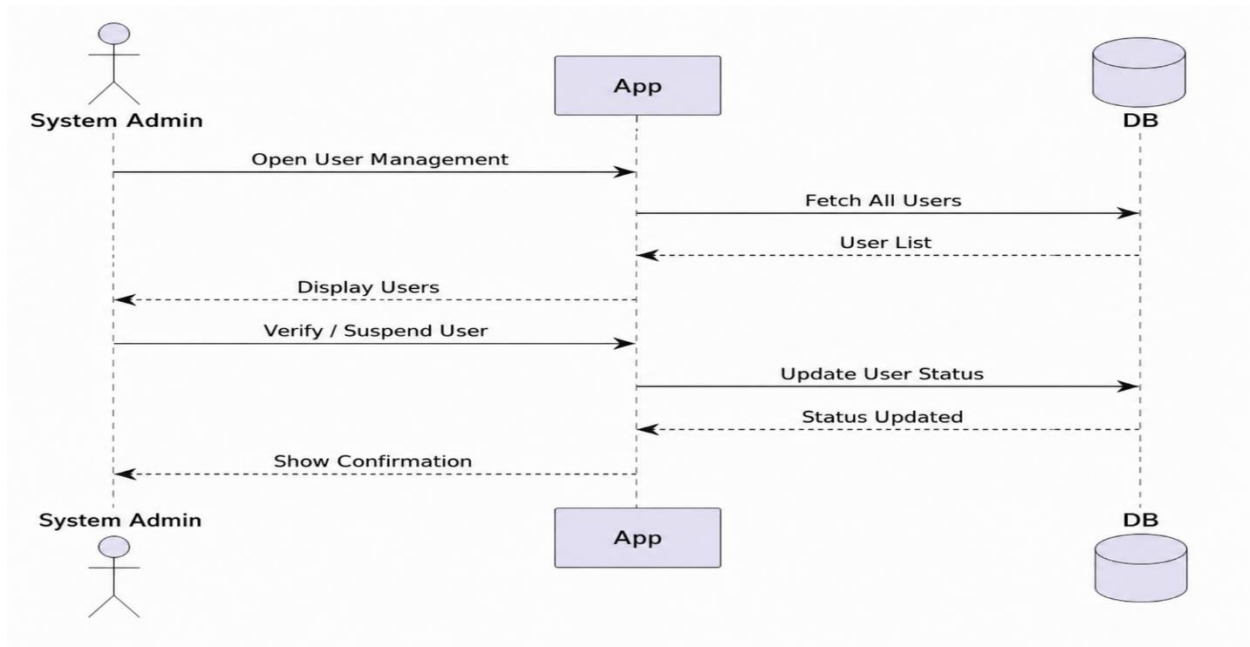


Figure 11 Sequence Diagram Manage User flow

3.2.3 State Transition Diagram

This diagram is used to show all major states through which the system gone through and all the major transitions

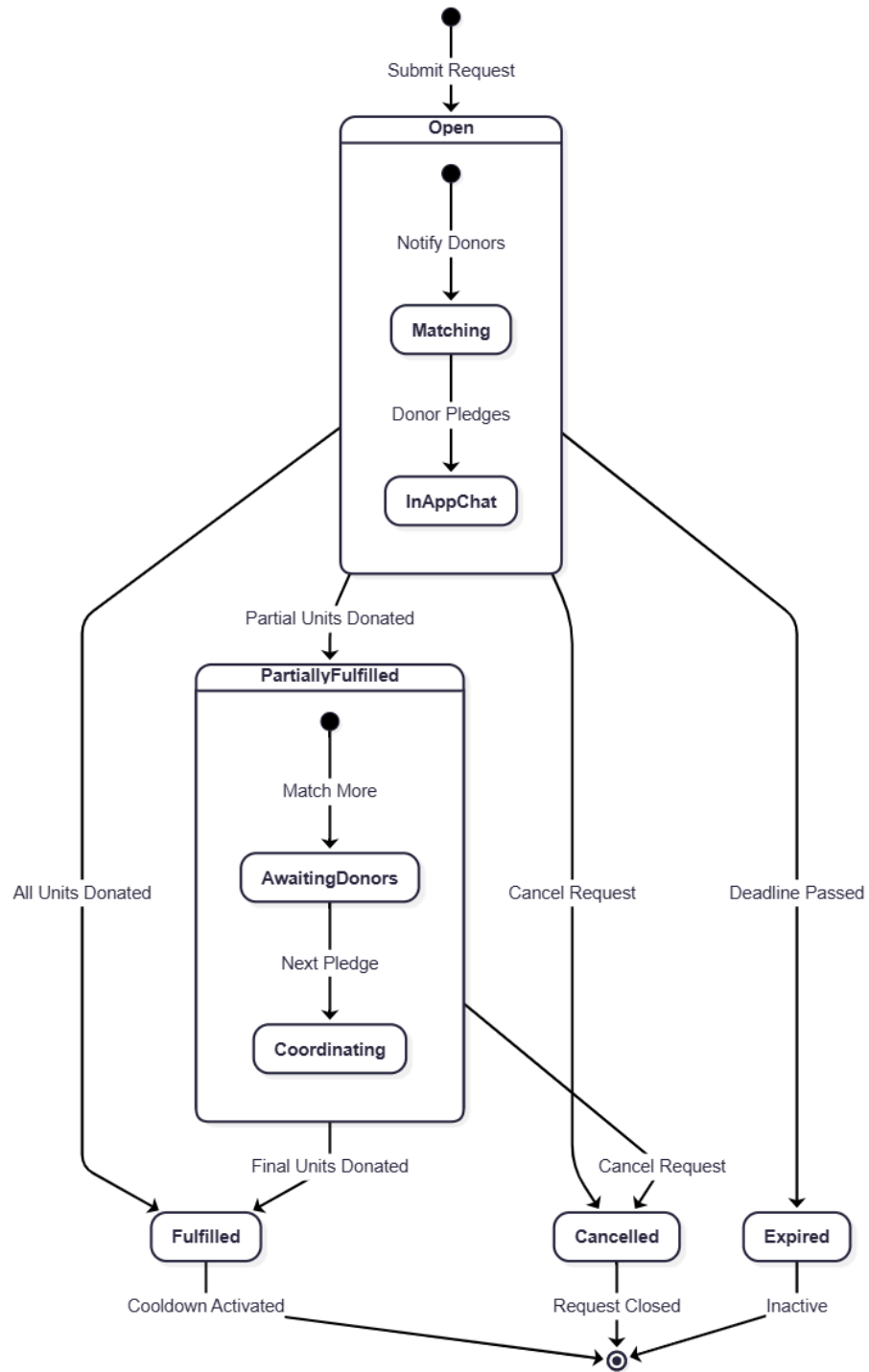


Figure 12 State Transition Diagram

3.3 Architectural Design

This section describes the architectural design of the Life Link Blood Donation System. It presents the major system components, explains how they interact, and shows how the overall functionality is achieved through collaboration among different subsystems. Life Link is developed using a Layered Client–Server Architecture combined with the MVC (Model–View–Controller) pattern. This approach improves maintainability, scalability, and separation of concerns.

3.3.1 Architecture Style

The system uses a Multi-tier Client–Server Architecture with the following layers:



Figure 13 Architecture Style

Table3.1: System Architecture Layers and Their Responsibilities

Layer	Components	Responsibility	Technologies
Presentation	Mobile App, Web Portal	User interface, input handling, display	React Native, Next.js
Business Logic	API Gateway, Services	Processing, validation, business rules	Node.js, NestJS
Data	Database, Storage	Data persistence, retrieval	Supabase (PostgreSQL)
External	Notification, Maps, Auth	Third-party integrations	OneSignal, Google Maps

3.3.2 UML Component Diagram

Diagram Explanation:

The component diagram shows how different parts of the system work together. Users interact with the Mobile App or Web Portal, which communicates with the backend through an API Gateway. The Business Layer contains specialized services for different functions, all of which store and retrieve data from the central database. External services provide additional capabilities like notifications, location, and authentication

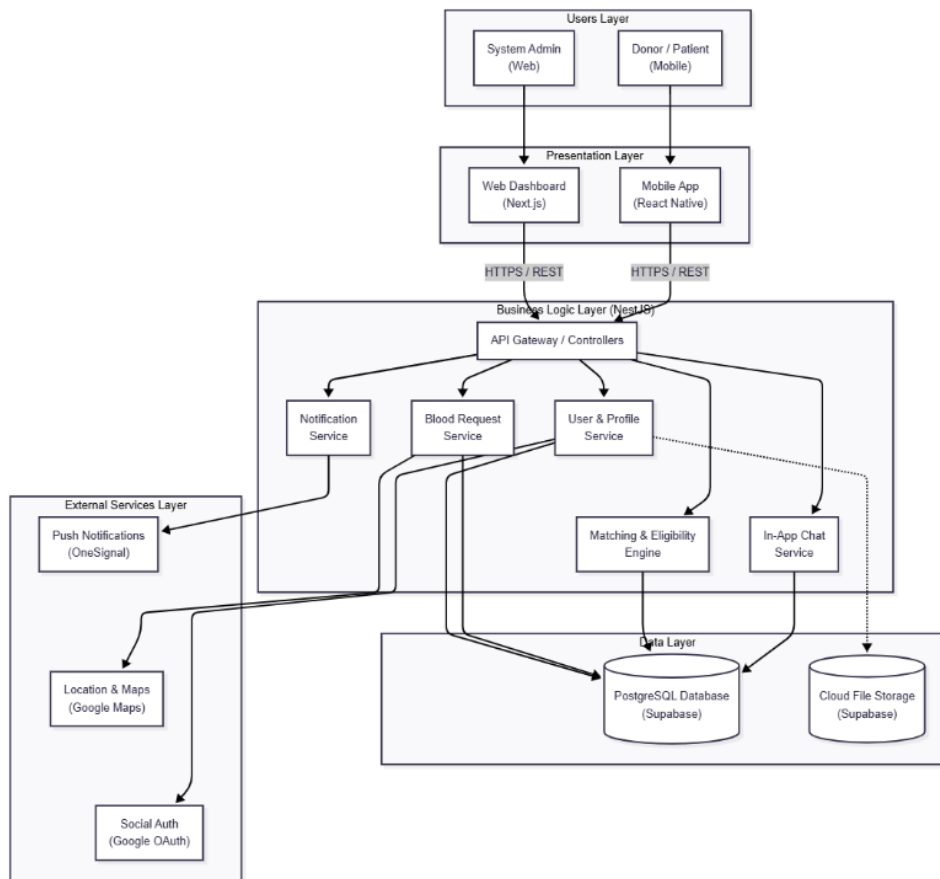


Figure 14 UML component diagram

3.3.3 Process Flow Diagram

The following diagram shows the end-to-end flow of how the system processes a blood request:

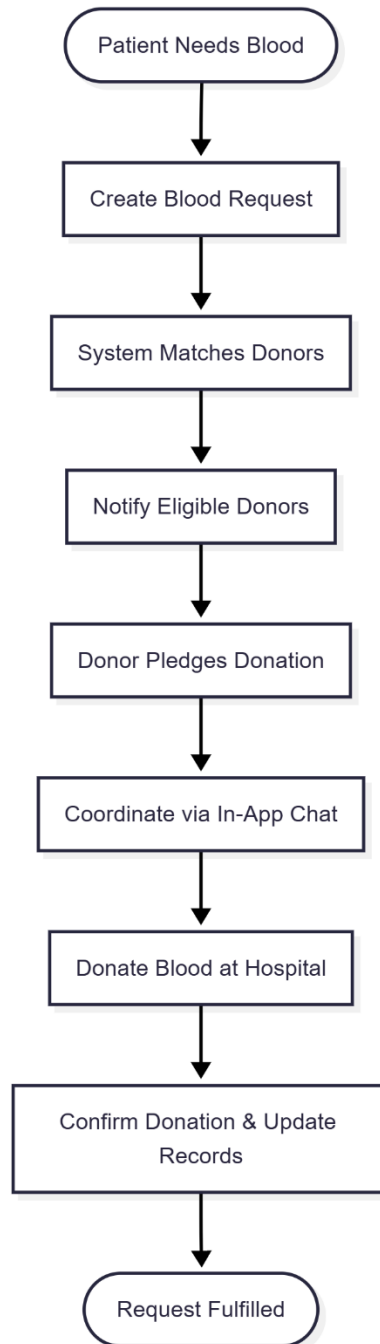


Figure 15 Process flow diagram

3.4 Data Design

This section explains how the information domain of the Life Link Smart Blood Donation System is transformed into data structures. It describes how major system entities are stored, processed,

and organized, including database design and other data structures used throughout the system

3.4.1 Database Schema Design

The system uses a relational database model with the following main tables and relationships:

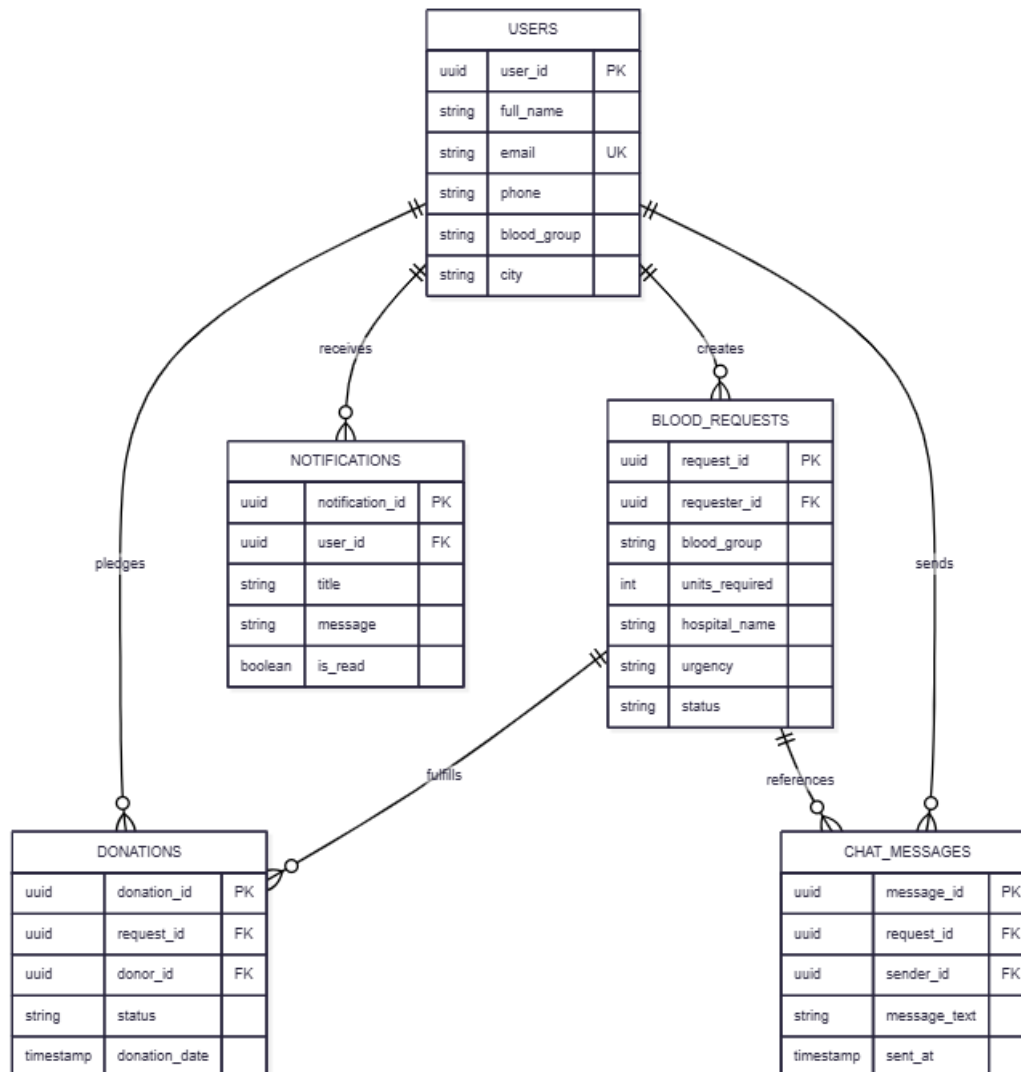


Figure 16 Database Design scheme

3.4.2 Data Dictionary

Alphabetical List of System Entities:

Table3.2: System Terminology and Data Dictionary

Terminology	Description
Admin	Object: Represents system administrator. Manages users, views analytics, and monitors requests.
availability	Boolean: Indicates whether a donor is currently eligible and available to donate.
bloodGroup	String: Blood type of donor or request (e.g., A+, B+, O-, AB+).
bloodRequestId	String: Unique identifier for each blood request record.
BloodRequest	Object: Stores patient blood demand details including required units, hospital, and urgency.
cancelRequest()	Method: Cancels an open or pending blood request.
ChatMessage	Object: Represents an in-app real-time message exchanged between a requester and a donor.
checkEligibility()	Method: Verifies donor eligibility based on 90-day cooldown, minimum age, and health checklist.
createdDate	Date: Timestamp when a record or blood request was created.
Donation	Object: Represents a pledged or completed blood donation record.
donationDate	Date: Date and time when the blood donation took place.
donationId	String: Unique identifier for a donation record.
Donor	Object: Role/subtype of User representing blood donors who pledge to fulfill requests.
eligibilityStatus	Boolean: Indicates whether the donor meets criteria to donate.
email	String: Registered email address used for login and identification.
generateReport()	Method: Generates administrative statistics and usage reports.
lastDonationDate	Date: Date of the donor's most recent blood donation.
location	String: Hospital address, city, or geographical coordinates for a request.
login()	Method: Authenticates a user into the system.
logout()	Method: Terminates the current active user session.
manageUsers()	Method: Administrative function to view, update, or ban users.
message	String: Content text of a system push alert or in-app notification.
Notification	Object: Represents automated push and in-app alerts sent to users.
password	String: Securely hashed password for user authentication.
Patient	Object: Role/subtype of User representing requesters in need of blood.
phone	String: Contact phone number of the user.
pledgeDonation()	Method: Allows a donor to commit to fulfilling an open blood request.
register()	Method: Registers a new user account in the system.
requestStatus	String: Current state of a blood request (open, partially_fulfilled, fulfilled, cancelled).
sendMessage()	Method: Sends a real-time chat message between requester and donor.
sendNotification()	Method: Dispatches push notifications to matched users via OneSignal.
status	String: General state indicator for entities (active, completed, cancelled).
unitsRequired	Integer: Total number of blood units requested for the patient.
updateProfile()	Method: Updates personal user details, blood group, or city.

urgency	String: Priority level of a blood request (normal, high, critical).
User	Object: Base entity representing all registered accounts in the system.
userId	String: Unique identifier assigned to each user.
viewHistory()	Method: Retrieves donation and request history for a user.

3.4.3 Object-Oriented Approach

1. User (Parent Class)

Table3.3: User Class Attributes and Methods

Attribute	Data Type
userId	String
name	String
email	String
phone	String
dob	Date
city	String
Method	Parameters
register()	name, email, password
login()	Email, password
updateProfile()	Phone, city, dob

2. Donor (inherits User)

Table3.4: Donor Class Attributes and Methods

Attribute	Data Type
bloodGroup	String
lastDonatedAt	Date
livesSaved	Integer
isEligible	Boolean
Method	Parameters
checkEligibility()	—

pledgeDonation (requestId)	requestId
-------------------------------	-----------

3. Patient (inherits User)

Table3.5: Patient Class Attributes and Methods

Attribute	Data Type
patientName	String
Method	Parameters
createBloodRequest()	bloodGroup, units, hospital, urgency
cancelRequest()	requestId

4. Admin (inherits User)

Table3.6: Admin Class Attributes and Methods

Attribute	Data Type
role	String
Method	Parameters
manageUsers()	userId, action
viewAnalytics()	—

5. Blood Request

Table3.7: Blood Request Class Attributes and Methods

Attribute	Data Type
requestId	String

bloodGroup	String
unitsRequired	Integer
fulfilledUnits	Integer
hospitalName	String
urgency	String
status	String
Method	Parameters
updateStatus()	Status
cancelRequest()	—

6. Donation

Table3.8: Donation Class Attributes and Methods

Attribute	Data Type
donationId	String
donationDate	Date
status	String
Method	Parameters
recordDonation()	donorId, requestId
confirmCompletion() ()	donationId

7. ChatMessage

Table3.9: Chat Message Class Attributes and Methods

Attribute	Data Type
messageId	String
requestId	String
senderId	String

messageText	String
sentAt	DateTime
Method	Parameters
sendMessage()	requestId, text
markAsRead()	threadId

8. Notification

Table3.10: Notification Class Attributes and Methods

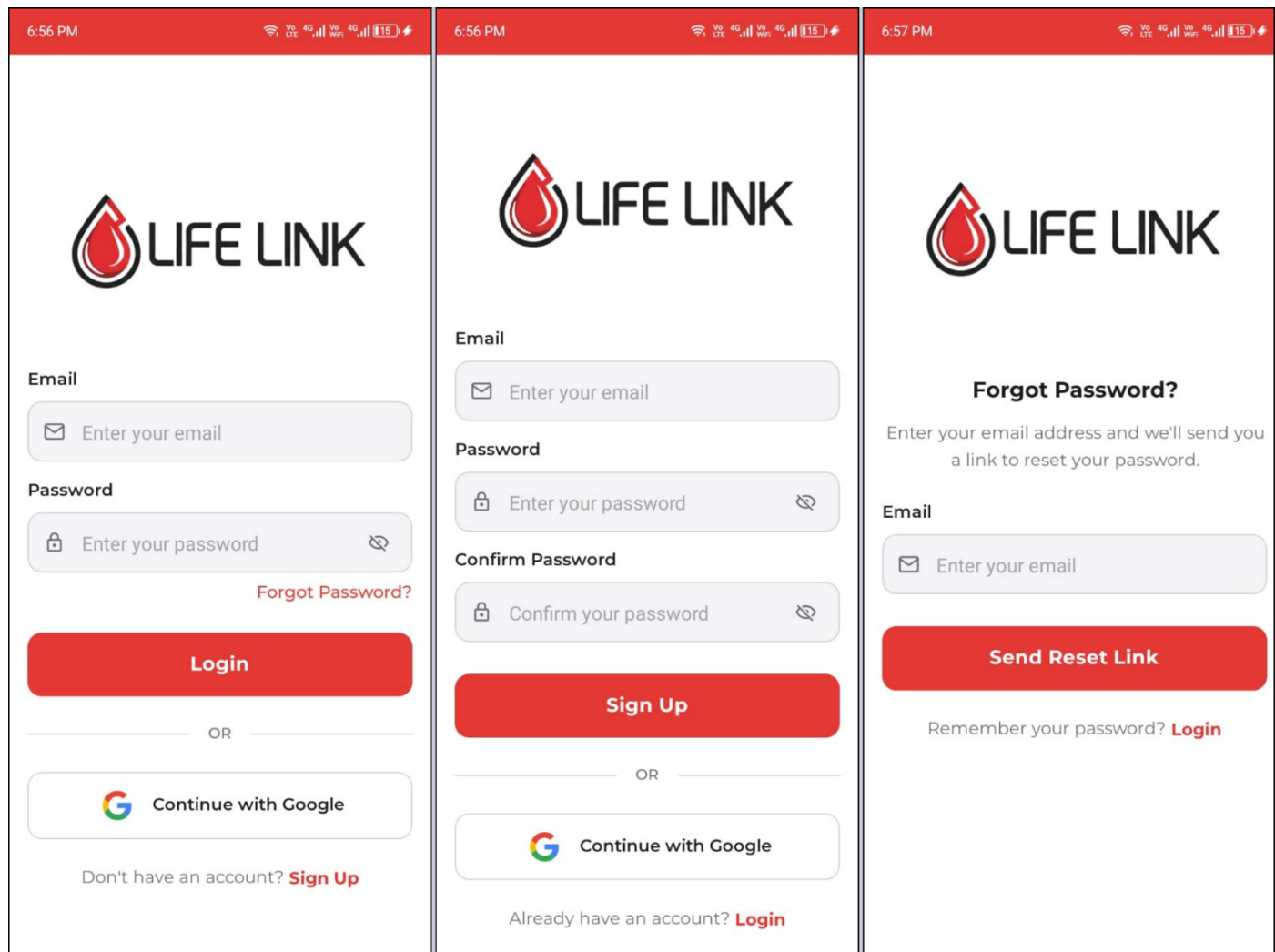
Attribute	Data Type
notificationId	String
message	String
isRead	Boolean
createdAt	DateTime
Method	Parameters
sendNotification()	userId, message
markAsRead()	notificationId

3.5 User Interface Design

The Life Link system provides an intuitive, bilingual (English/Urdu) interface designed for ease of use across all user types. From the user's perspective, the system facilitates the entire blood donation process through a mobile-first approach with clear navigation, real-time feedback, and simple workflows

3.5.1 Screen Images

Figure 17 Auth Screens



Request Blood

Blood Type

A+

A-

B+

B-

O+

O-

AB+

AB-

Quantity

Select units needed (1-10)

Urgency Level

Emergency

Normal

Location

City or area

Hospital Name

Enter hospital name

Notes

Any special instructions or details...

Submit Request

Save Draft

Track Request

URGENT • A+

#B-92841

Request ID

Status History

Submitted

10:00 AM • Request received

Matching

10:15 AM • Search initiated

Accepted

10:45 AM • Donor confirmed

Scheduled

Pending confirmation

Completed

Pending

Matched Donor

ON THE WAY

2.4 km away

Sarah Jenkins

4.9 • 12 Donations

Contact Support

Cancel Request

Schedule Appointment

Selected

Red Cross Center - Downtown

123 Medical Plaza, Suite 100

1.2 mi away • Open until 6 PM

Select Date

October 2023

October 2023

5

6

7

8

9

10

11

12

13

Available Time

MORNING

09:00 AM

09:30 AM

10:00 AM

10:30 AM

11:00 AM

11:30 AM

Confirm Appointment

Cancel

Please arrive 15 minutes before your scheduled time to complete the necessary health screening.

Figure 18 Request Screens

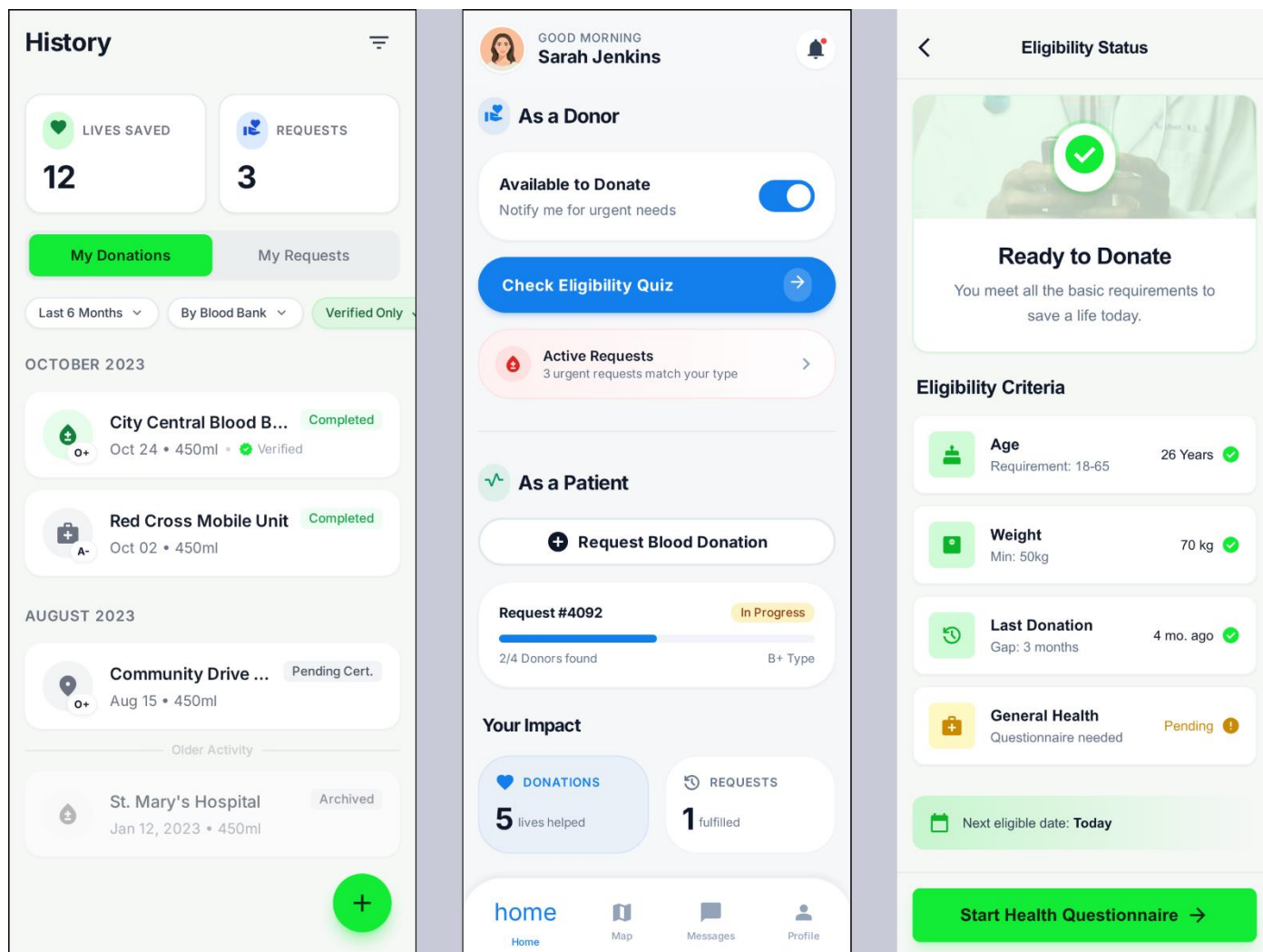


Figure 19 User's Screens

3.6 Design Decisions

For system architecture, we used a multi-tier Client–Server architecture with the MVC pattern because it separates UI, business logic, and data layers, making the system easy to manage and maintain. React Native and Next.js were used for the frontend, while NestJS was used for backend services. For database design, we used PostgreSQL through Supabase because the system handles structured relational data such as users, blood requests, donations, and chat messages. Supabase also provides built-in authentication and real-time features. Object-oriented design patterns were applied to improve code organization: the Factory Pattern was used for creating different user roles, while the Singleton Pattern was used for core services to avoid duplicate instances. A rule-based approach with location-based matching was used to identify eligible donors and prioritize nearby donors for faster response. For security, Supabase Authentication with JWT and Role-Based Access Control (RBAC) was used to ensure secure access based on user roles.

For notifications and real-time updates, OneSignal and Supabase Realtime were used to deliver instant alerts and live status changes. React Native was used for mobile app development and Next.js for the admin panel to ensure fast development and a consistent user experience. Vercel, Supabase, and GitHub were used for deployment, backend services, and version control respectively.

3.7 Summary

Chapter 3 describes the overall design of the Life Link blood donation system, including its assumptions, limitations, and layered architecture. It explains how the user interface, backend logic, database, and external services work together, supported by diagrams showing data flow. The chapter also covers data organization, a simple bilingual interface, and technology choices made to ensure the system is secure, scalable, and ready for development.

Chapter 4

Implementation

4. Implementation

4.1 Algorithm:

This chapter explains how the system is implemented. It describes the working of main modules using simple algorithms and explains the external APIs and tools used in the system.

1. Algorithm 1: User Registration

Table 4.1: User Registration

Name	User Registration
Input:	Email, Password, Confirm Password (or Google OAuth token)
Output:	New user account created with session token
Steps:	1. Start 2. User enters email, password, and confirm password on the Sign Up screen 3. System validates password length and matching confirmation 4. System sends registration request to backend auth service 5. If email already exists, system displays "Email already registered" error message and stops 6. If email is unique, Supabase hashes password and creates user record 7. System displays success alert and redirects user to Onboarding screen to set full name, blood group, date of birth, and city 8. End

2. Algorithm 2: User Login:

Table 4.2: User Login

Name	User Login
Input:	Email, password
Output:	Authenticated session with JWT token
Steps:	1. Start 2. User opens the login screen and enters email and password 3. System performs client-side field validation (email format, non-empty

	password) 4. System submits credentials to Supabase Auth service 5. If credentials are valid, system securely stores session tokens, updates app state, and navigates to the Home Dashboard (or Onboarding if profile is incomplete) 6. Else (if credentials are invalid), system displays an "Invalid email or password" error message 7. End
--	--

3. Algorithm 3: Blood Request Creation

Table 4.3: Create Blood Request

Name	Blood Request Creation
Input:	blood_group, units_required, hospital_name, city_id/coordinates, urgency (Normal, High, Critical)
Output:	New blood request stored with status 'open' and matching broadcast triggered
Steps:	1. Start 2. User opens the "Request Blood" screen 3. User enters required blood group, units needed, hospital name, city, and urgency level 4. System validates form inputs (valid blood type, units, hospital name) 5. System submits data to backend API and saves record in database with status 'open' 6. System triggers rule-based donor matching engine to identify eligible donors 7. System displays "Request created successfully!" confirmation and navigates to the request details 8. End

4. Algorithm 4: Donor Matching

Table 4.4: Donor Matching

Name	Donor Matching
Input:	blood_group, request location coordinates / city
Output:	Ordered list of compatible, eligible donor IDs
Steps:	1. Start 2. Matching service receives the newly created blood request 3. System queries the users table for active users with compatible ABO/Rh blood groups 4. System filters out users who are ineligible (under 18 years old or currently on the 90-day donation cooldown) 5. For each remaining eligible donor, the geodesic distance to the hospital location is calculated using the Haversine formula 6. Donors located within the same city or closest radius are sorted by ascending distance 7. System selects the top matched donors and passes their IDs to the Notification Service 8. End

5. Algorithm 5: Notification Sending

Table 4.5: Notification Sending

Name	Notification Sending
Input:	List of matched donors
Output:	Push notifications delivered (or logged as failed)
Steps:	1. Start 2. System receives matched donors 3. For each donor: 4. Prepare notification message 5. Send notification via OneSignal (or the configured push-service) 6. Log notification status (sent or failed) 7. Display "Notifications Sent"

6. Algorithm 6: Search and Filter Blood Requests

Table 4.6: Search and Filter Blood Requests

Name	Search and Filter Blood Requests
Input:	Filter criteria (blood_group, urgency, city, distance radius)
Output:	Filtered list of active blood requests displayed on feed
Steps:	1. Start 2. User navigates to the Blood Requests Feed screen 3. User selects filter parameters (e.g., Blood Group: O+, Urgency: High/Critical, Distance: < 15 km, or City) 4. System queries active blood requests from the database matching the criteria 5. System calculates proximity distance from user's GPS coordinates to each hospital using the Haversine formula 6. System displays request cards showing patient name, hospital, units required, distance, and urgency badge 7. User taps a card to view request details, verify eligibility checklist, and coordinate via In-App Chat 8. End

7. Algorithm 7: Donor Eligibility Verification

Table 4.7: Donor Eligibility verification

Name	Donor Eligibility Verification
Input:	Donor ID, Date of Birth, last_donated_at, pre-donation checklist answers
Output:	Eligibility status (Eligible or Cooldown) and commitment confirmation
Steps:	1. Start 2. System calculates donor age from Date of Birth: $\text{Age} = \text{CurrentDate} - \text{DOB}$ 3. If $\text{Age} < 18$, system marks donor permanently ineligible and stops 4. System checks elapsed days since last donation: $\text{DaysElapsed} = \text{CurrentDate} - \text{last_donated_at}$ 5. If $\text{DaysElapsed} < 90$, system displays Cooldown status with remaining

	days and next eligible date, then stops 6. If donor is eligible and attempts to pledge for a request, system displays the interactive Health Checklist modal 7. Donor confirms weight ≥ 50 kg, no recent illness, and no recent antibiotic use 8. If any checklist condition fails, pledge is rejected; if all pass, donation pledge is confirmed 9. End
--	---

4.2 External APIs/SDK

Name	Description	Purpose	Usage
Supabase API	Backend-as-a-service that provides PostgreSQL storage, authentication, row-level security and realtime listeners.	Stores user profiles, donor data, blood-request records and matches; handles sign-up / sign-in.	Manage user authentication and store and retrieve user profiles, donor information, blood requests, and matching data
OneSignal API	Push-notification Service for mobile Apps.	Sends real-time alerts to matched donors when a new blood request is created.	Send push notifications and real-time alerts to donors about new blood requests
Google Maps API	Native map component that wraps the Google Maps SDK on Android and iOS.	Visualizes request locations, lets users pick a location, and enables distance calculations for donor-matching.	Display maps, select and view blood request locations, and calculate distances for

			location-based donor matching
Google Sign-In API	OAuth 2.0 authentication SDK for Google identity integration (@react-native-google-signin/google-signin).	Provides quick, password less social authentication for mobile users.	Handles one-tap sign-up and login, exchanging Google ID tokens with Supabase for secure session creation.

4.3 Code Repository

The project source code is managed using Git for version control and is hosted on GitHub. All project files, including mobile application code, web admin panel, and backend services, are stored in the repository to ensure proper tracking of changes and smooth collaboration among team members.

Component	Repository Link
Mobile Application	https://github.com/Usman22209/LifeLink
Web Admin Dashboard	https://github.com/hadia41/Web_dashboard
Backend API Services	https://github.com/Usman22209/life-link-backend

4.4 Summary

The implementation follows clear algorithms for user registration, login, blood request creation, donor matching, notification dispatch, blood request feed filtering, and donation fulfillment. It integrates Supabase for data and authentication, OneSignal for real-time push alerts, and Google Maps via react-native-maps for location and proximity handling. All files are tracked in the GitHub repository, enabling collaborative development, version history, and issue tracking, while the repository README provides the necessary setup and execution guidance.

Chapter 5

Testing and Evaluation

5. Introduction

This chapter presents the testing and evaluation strategy for the LifeLink Smart Blood Donation System. Validation and verification activities cover all major components, employing both manual and automated techniques. Unit, functional, and integration tests are executed to confirm that the system satisfies its requirements. Test cases are recorded using a consistent tabular format.

5.1 Unit Testing (UT):

This section presents unit test cases that verify individual form validations, authentication logic, and component behaviors in isolation.

5.1.1 Test Case 1

Table 5.1 Unit Test Case 1

Field	Description
Test case ID	UT-1
Title	User Sign-Up Form Validation
Description	Verify that the sign-up form validates required fields and password constraints.
Objective	To ensure incomplete or invalid credentials are not submitted to the authentication service.
Driver/Precondition	Mobile app is running and Sign-Up screen is open.
Test Steps	1. Leave email field empty. 2. Enter a password shorter than 6 characters. 3. Tap Sign Up button.
Input	Empty email, password = 123

Expected Results	System displays validation error messages: Email is required and Password must be at least 6 characters.
Actual Result	Error messages displayed correctly below inputs.
Remarks	Pass

5.1.2 Test Case 2

Table 5.2 Unit Test Case 2

Field	Description
Test case ID	UT-2
Title	Blood Request Form Validation
Description	Verify that the blood request form enforces required hospital and unit fields.
Objective	To ensure incomplete blood requests cannot be created.
Driver/Precondition	User is logged in and Request Blood screen is open.
Test Steps	1. Leave hospital name empty. 2. Select blood group. 3. Enter units = 0. 4. Tap Submit.
Input	Empty hospital name, units = 0, blood group = "B+".
Expected Results	System displays error messages: "Hospital name is required" and "Units must be at least 1".
Actual Result	Form validation errors displayed correctly.
Remarks	Pass

5.1.3 Test Case 3

Table 5.3 Unit Test Case 3

Field	Description
Test case ID	UT-3
Title	Login Credentials Validation

Description	Tests client-side validation for email and password formatting.
Objective	To prevent malformed inputs from reaching the authentication server.
Driver/Precondition	Login screen is displayed.
Test Steps	1. Enter invalid email format. 2. Tap Login button.
Input	Invalid email string, non-empty password.
Expected Results	System displays error: "Please enter a valid email address".
Actual Result	Error message displayed correctly
Remarks	Pass

5.2 Functional Testing (FT)

Functional testing verifies that complete user-level features—including request creation, feed filtering, and profile updates—operate correctly from end to end.

5.2.1 Test Case 1

Table 5.4 Functional Test Case 1

Field	Description
Test case ID	FT-1
Title	User Registration and Profile Onboarding.
Description	Validate that a new user can sign up and complete their onboarding profile.
Objective	Ensure new accounts are created and profile records are stored in Supabase.
Driver/Precondition	Internet connection active, Sign-Up screen open.
Test Steps	1. Enter valid email and password. 2. Submit form. 3. On Onboarding screen, enter name, DOB, blood group, and city. 4. Tap Complete.
Input	Valid email, password, full name, DOB (age ≥ 18), blood group = "O+", city = "Lahore".

Expected Results	Account created in Supabase Auth, profile stored in users table, user navigated to Home Dashboard.
Actual Result	Registered successfully and navigated to dashboard.
Remarks	Pass

5.2.2 Test Case 2

Table 5.5 Functional Test Case 2

Field	Description
Test case ID	FT-2
Title	Blood Request Submission
Description	Verify that a requester can create and post an active blood request.
Objective	Ensure blood requests are saved and broadcasted to matching donors.
Driver/Precondition	User is logged in with a completed profile.
Test Steps	1. Open "Request Blood" screen. 2. Select blood group (A+), units (2), hospital name, and urgency (High). 3. Tap Submit.
Input	Blood group = "A+", units = 2, hospital = "Mayo Hospital", urgency = "High", city = "Lahore".
Expected Results	Request record created with status 'open', confirmation toast displayed, and donor matching initiated.
Actual Result	Request created successfully and visible on active feed.
Remarks	Pass

5.2.3 Test Case 3

Table 5.6 Functional Test Case 3

Field	Description
Test case ID	FT-3
Title	User Login Authentication
Description	Confirm that a registered user can log-in and obtain a valid session.

Objective	Ensure authorized users gain access to their personalized dashboard.
Driver/Precondition	User is already registered.
Test Steps	1. Open login screen. 2. Enter registered credentials. 3. Tap Login.
Input	Valid Email and password
Expected Results	Session token returned, user authenticated, and redirected to Home screen.
Actual Result	Login successful, Home screen rendered with user statistics.
Remarks	Pass

5.2.4 Test Case 4

Table 5.7 Functional Test Case 4

Field	Description
Test case ID	FT-4
Title	Filter Blood Requests Feed
Description	Test the search and filter capabilities on the public blood requests feed.
Objective	Ensure the system filters active blood requests by blood group and urgency level.
Driver/Precondition	Active blood requests exist in the database; user opens Feed screen.
Test Steps	1. Navigate to Feed screen. 2. Tap Urgency filter and select "High". 3. Select Blood Group "B+". 4. Observe displayed list.
Input	Urgency = "High", Blood Group = "B+".
Expected Results	Feed dynamically updates to show only active requests matching B+ with High urgency.
Actual Result	Matching request cards displayed correctly with accurate badges.
Remarks	Pass

5.2.5 Test Case 5

Table 5.8 Functional Test Case 5

Field	Description
Test case ID	FT-5
Title	Update User Profile
Description	Verify that a user can modify their contact and profile information.
Objective	Ensure profile updates are persisted in the database.
Driver/Precondition	User is logged in and opens Profile screen.
Test Steps	1. Tap Edit Profile. 2. Update phone number and city. 3. Tap Save.
Input	Updated phone number, updated city.
Expected Results	Profile record updated in database, success toast shown.
Actual Result	Data updated in system and reflected on profile screen.
Remarks	Pass

5.3 Integration Testing (IT)

Integration testing verifies that the mobile client, NestJS backend services, Supabase database, and third-party APIs (OneSignal, Google Maps) communicate and exchange data reliably.

5.3.1 Test Case 1

Table 5.9 Integration Test Case 1

Field	Description
Testcase ID	IT-1
Title	User Registration with Supabase Authentication & Database
Description	Verify end-to-end data flow from client registration UI to Supabase database.
Objective	Ensure user credentials and profile records are properly linked across auth and database tables.
Driver/Precondition	Registration screen open, Supabase service reachable.

Test Steps	1. Submit valid sign-up form. 2. Complete onboarding profile. 3. Check Supabase Auth and public.users table.
Input	Valid user registration and profile payload.
Expected Results	Auth record created in auth.users, linked profile row inserted in public.users.
Actual Result	User record persisted with matching UUID foreign key.
Remarks	Pass

5.3.2 Test Case 2

Table 5.10 Integration Test Case 2

Field	Description
Test case ID	IT-2
Title	User Login with Token Storage
Description	Tests integration between Login UI, Supabase Auth service, and device Keychain/AsyncStorage.
Objective	Ensure JWT access and refresh tokens are securely received and stored on the device.
Driver/Precondition	User account exists in database.
Test Steps	1. Enter email and password. 2. Tap Login. 3. Verify session storage.
Input	Valid login credentials.
Expected Results	Backend returns JWT session tokens; mobile app persists tokens in secure storage and attaches token to subsequent API headers.
Actual Result	Login successful, tokens persisted, authenticated API requests succeed.
Remarks	Pass

5.3.3 Test Case 3

Table 5.11 Integration Test Case 3

Field	Description
Test case ID	IT-3
Title	Blood Request Creation with Donor Notification Dispatch
Description	Validate end-to-end path from request submission to OneSignal push alert delivery.
Objective	Ensure blood request creation triggers matching engine and dispatches push alerts to eligible donors.
Driver/Precondition	Matching eligible donors exist in the target city with notifications enabled.
Test Steps	1. Requester submits a High urgency blood request. 2. Backend saves request. 3. Matching engine identifies donors. 4. OneSignal API dispatches notification.
Input	Blood request payload (Blood Group = "O+", Urgency = "High", Hospital = "Jinnah Hospital")
Expected Results	Request stored with status 'open', OneSignal API returns 200 OK, notification banner appears on matching donor device.
Actual Result	Request saved and push notification delivered within <= 3 seconds.
Remarks	Pass

5.3.4 Test Case 4

Table 5.12 Integration Test Case 4

Field	Description
Test case ID	IT-4
Title	Blood Request Feed and Distance Calculation Integration
Description	Verify integration between mobile GPS location service, Haversine distance utility, and Supabase request queries.
Objective	Ensure active blood requests are fetched and displayed with accurate calculated distances.

Driver/Precondition	Device location permission granted; active requests present in database.
Test Steps	1. User opens Feed screen. 2. Client obtains device GPS coordinates. 3. Queries open requests from Supabase. 4. Computes geodesic distance to each hospital.
Input	Device GPS coordinates (lat, lon)
Expected Results	Feed displays request cards with correct calculated distance in kilometers (e.g., "2.4 km away").
Actual Result	Data displayed correctly with accurate distance calculations.
Remarks	Pass

5.4 Summary

This chapter detailed the verification and evaluation strategy for the Life-Link Smart Blood Donation System. Unit test cases confirmed that client-side form validations, input constraints, and authentication controls function reliably. Functional test cases verified core user workflows including registration onboarding, blood request posting, feed filtering, and profile updates. Finally, integration test cases confirmed robust data flow and synchronization between the React Native frontend, Supabase database, and the OneSignal notification service. All test suites executed successfully with zero blocking defects.

Chapter 6

System Conversion

6 System Conversion

System conversion is the process of shifting from a manual or old system to a new digital system. The Life-Link Blood Donation System replaces the old paper-based, phone call driven process with a fully digital mobile and web solution. This shift eliminates manual recordkeeping, reduces errors, and makes the platform easy for donors and patients to adopt. A proper system conversion helps in reducing problems, maintaining correct data, and making it easier for users to adopt the new system.

6.1 Conversion Method

For the implementation of the Life Link Smart Blood Donation System, the selected conversion method is Pilot Conversion.

6.1.1 Pilot Conversion:

The project employs a pilot conversion approach: a limited group of users (IT staff, selected students, and a few donors) receives the new system first. During the two-week pilot the core modules Login/Registration, Blood Request Creation, Donor Matching, and Notification sending are exercised in real world conditions. Feedback is gathered, issues are fixed, and the system is

refined before a full roll out to all users.

Justification:

Reduced Risk: Only a small number of users are involved at the start, so any problems will not affect all users.

Focused Testing: Important features like user registration, request creation, donor matching, and notifications can be tested properly in real use.

User Feedback: Early users can share their experience and suggest improvements, which helps in making the system better.

Pilot Scope:

Initial Group: Selected students and staff from the IT department and a small number of donors.

Modules Tested: Login/Registration, Request Creation, Donor Matching, and Notifications.

Duration: The system is tested for about two weeks, and after improvements, it is launched for all users.

6.2 Deployment

Deployment moves the Life-Link application from a development environment to a production ready system that end-users can download and use. It involves building the mobile client, publishing the backend services, configuring Supabase, setting up OneSignal, and verifying that everything works on real devices.

1.2.1 Deployment Steps:

Table 6.1 Deployment Steps

Step No.	Task	Details
1	Build Mobile APK	Generate a signed Android APK with the React-Native CLI/Gradle. The APK is then shared with pilot users via a secure download link.
2	Deploy Backend API	The NestJS server is pushed to a cloud VM and all REST endpoints are tested for accessibility.
3	Deploy Admin Dashboard	The Next.js admin panel is published on Vercel environment variables for Supabase and OneSignal are set there.

4	Configure Supabase	Database tables, authentication settings, and row-level-security policies are created and made live.
5	OneSignal Setup	Configure OneSignal push notification service and link it to the mobile client and backend are linked to the service.
6	Device Testing	The APK is installed on a range of Android devices (API 24 – 36) to verify UI layout, performance, and notification receipt.
7	Deployment Confirmation	Pilot users run user-acceptance tests; every core module (login, request creation, matching, notifications) must work in the live system.

1.2.2 Data Conversion

Table 6.2 Data Conversion

Step	Task	Details
1	Data Preparation	Collect donor and blood bank records from paper logs and spreadsheets
2	Data Cleaning	Remove duplicate entries, fix formatting issues, and verify phone, Email and contact information.
3	Data Transformation	Convert the collected data into the correct format for Supabase database tables.
4	Data Loading	Use Supabase for web console or custom upload scripts to populate the tables.

6.2.3 Training

Training materials were prepared to help donors and users understand how to use the Life Link application. The following training instructions are provided.

Instruction 1:

Table 6.3 Training Instruction 1

Step	Instruction
1	Register using your email address or log in with an existing account.
2	Complete your profile by entering your blood type, city, and contact number.
3	For donors: Browse active blood requests on the Home screen.
4	Tap a request that matches your blood type to view full details.

5	Accept the request and schedule a donation appointment at the nearest hospital.
---	---

Instruction 2:

Table 6.4 Training Instruction 2

Step	Instruction
1	Log in as a patient and open the Create Request screen.
2	Enter the required blood type, number of units, urgency level, and hospital location.
3	Submit the request and wait for the system to match you with available donors.
4	Receive a notification when a donor confirms an appointment.
5	View the request status in real time from your home screen.

1.3 Post Deployment Testing

Post-deployment testing was carried out to confirm that the Life Link system is correctly installed, fully configured, and operating as expected in the live environment. The goal was to ensure data integrity, feature functionality, and a smooth user experience after going live.

6.3.1 Post Deployment Testing Steps:

Table 6.5 Post Deployment Testing Steps

Step	Test Activity	Expected Outcome
1	Launch Application	App starts without errors on Android devices.
2	Login Functionality	Users can log in using email and password successfully.
3	Blood Request Creation	Patient can create a request, and it appears in the database.
4	Donor Notification	Matched donors receive OneSignal push notifications on their devices.
5	Appointment Scheduling	Donor can schedule an appointment and request status updates.
6	Language Switch	App correctly switches between English and Urdu.
7	Data Integrity Verification	User profiles, requests, and donation records persist correctly after logout and login.
8	Admin Panel Access	Admin can log into the web dashboard and manage users and requests.
9	Performance Check	App responds within acceptable time on various Android devices.

1.4 Challenges and Solutions

During deployment and post-deployment phases of Life Link Smart Blood Donation System several technical and operational challenges were encountered. Below is a list of the main challenges along with their applied solutions.

Table 6.6 Challenges and Solutions

Challenge	Description	Solution Implemented
Push Notification Delivery	Some Android phones block notifications due to aggressive battery optimization.	Users were guided to disable battery optimization for LifeLink and enable OneSignal fallback delivery.
Supabase Row-Level Security	Incorrect security rules blocked certain users from reading or writing data.	Security rules were reviewed and rewritten for each user role, then tested thoroughly before redeployment.
APK Installation on Restricted Devices	University issued devices refused installation of unsigned APKs.	Built a signed APK with a valid certificate via Expo EAS and distribute it securely.
Data Format Issues During Migration	Some manual records contained missing or inconsistent fields that caused errors during upload.	Ran a data-transformation script that validated and corrected fields before bulk loading into Supabase.
Device Compatibility	UI layouts appeared broken on some older Android devices.	Responsive design patterns and flexible layout constraints were applied. Testing was extended to devices running API 24 to 36.
User Onboarding Confusion	Some pilot users were unsure about how to navigate the app.	Produced a step-by-step tutorial video (shared on WhatsApp) and added in-app tooltips for first-time user.

1.5 Summary

This chapter explains how the Life-Link was moved from development to production. The process covered building and distributing a signed Android APK, deploying the NestJS backend and Next.js admin panel, configuring Supabase and OneSignal, converting legacy donor data, and training end-users. Post-deployment testing confirmed that all core features login, request creation, donor matching, notifications, language switching, and admin

management function correctly. The challenges encountered (notification delivery, security rules, APK signing, data migration, and UI compatibility) were resolved with the measures listed above, ensuring a stable, user-ready system.

Chapter 7

Conclusion

2. Introduction

This chapter summarizes what has been finished, what is only partially done, and what remains to be developed.

7.1 Evaluation

The table below lists all the objectives that were defined in Chapter 1 and shows whether each one has been achieved in the final system.

7.1.1 Project Objectives:

Table 7.1: Evaluation of Project Objectives

Objectives	Status
Develop a mobile application for blood donors and patients.	Completed
Allow patients to create emergency and normal blood requests easily.	Completed
Notify suitable donors about blood requests through instant push notifications.	Completed
Check donor eligibility based on basic medical rules such as age, weight, and last donation date.	Completed
Maintain donation history and update donor availability automatically.	Completed
Provide a simple admin panel to manage users, blood requests, and system data.	Completed
Ensure basic security using user authentication and access control.	Completed

2.2 Traceability Matrix

The traceability matrix below shows how each functional requirement from the Software Requirements Specification (SRS) is connected to the design, source code, and test cases that verify it. This ensures that all requirements have been properly implemented and tested in the final system.

Table 7.2: Requirements Traceability Matrix

Req. ID	Requirement Description	Design Specification	Code	Test ID
FR-1	User Registration and Login	Auth flow, Supabase Auth integration	AuthScreen.tsx, AuthService.ts	UT-1, FT1, IT1
FR-2	Blood Request Creation	Request data model, request-service AP	CreateRequest.tsx, requestService.ts	FT2, IT2
FR-3	Donor Eligibility Check	Eligibility check logic, Donor (age, weight, last donation	EligibilityService.ts	UT3, IT3

FR-4	Donor Matching and Notifications	Matching algorithm, OneSignal notification wrapperr	MatchingService.ts, notifyService.ts	UT4, FT3, IT2
FR-5	Blood Donation Fulfillment	Request status updates, donation record handling	donationService.ts, requestService.ts	FT4, IT4
FR-6	Donation History and Tracking	Donation History component, Donor profile view	HistoryScreen.tsx, donationService.ts	FT5
FR-7	Bilingual User Interface (English / Urdu)	Language context, i18n strings files	LanguageContext.tsx, strings-en.ts, strings-ur.ts	FT6, IT5
FR-8	User Profile Management	Profile screen, profile-service API	ProfileScreen.tsx, userService.ts	FT7
FR-9	Administrative User Management	Admin dashboard, User management service	AdminUsers.tsx, adminService.ts	FT8, IT6

2.3 Conclusion

The Life Link Smart Blood Donation System was built to close a critical gap in Pakistan's healthcare delivery by offering a fast, reliable, and accessible way to connect donors with patients. The solution integrates a React Native mobile app, a Next.js admin dashboard, and a NestJS backend hosted on Supabase. All core features user registration, blood-request creation, donor matching, appointment scheduling, donation confirmation, bilingual UI, and real-time push notifications have been implemented and verified through unit, functional, and integration tests. Pilot-user feedback confirms that the app is easy to navigate and dramatically shortens the time needed to locate compatible donors. The project demonstrates that modern mobile technology combined with a real-time data backend can make a tangible difference in healthcare accessibility across Pakistan.

2.4 Future Works

After completing the current version of Life Link, the team plans to extend the system with the following features and improvements. A web-based version of the donor and patient portal will be created to support users without smartphones. Payment gateway integration will be added to allow blood banks to charge for processing fees in a transparent manner. Advanced analytics and reporting features will be added to the admin dashboard to help track donation trends and shortage forecasts. The AI-based donor recommendation model will be improved using a larger dataset to make matches more accurate. Multi-city and regional expansion support will be built in, allowing the system to scale across Pakistan and potentially to other countries. Integration with hospital information systems will also be explored to further automate the blood request and fulfillment process.

References

- Cross, A. R. (2023). *Red Cross Blood Donor App*. Retrieved from American Red Cross: <https://www.redcrossblood.org/blood-donor-app.html>
- Li, L. (2023). Mobile applications for encouraging blood donation: A systematic review. *Journal of Health Informatics*. Retrieved from <https://www.ncbi.nlm.nih.gov/pmc/articles/PMC10563464/>
- Organization, W. H. (2021). *Global report on blood safety and availability*. Retrieved from World Health Organization: <https://www.who.int/publications/i/item/9789240034231>
- Pakistan, E.-B. (2024). *E-Blood Pakistan – Online Blood Donation Platform*. Retrieved from EBLOOD: <https://www.eblood.com.pk>
- TensorFlow. (2023, April 17). *On-device AI with TensorFlow Lite*. Retrieved from TensorFlow Blog: <https://blog.tensorflow.org/2023/04/on-device-ai-with-tensorflow-lite.html>